Advanced Computer-Aided VLSI System Design

Clock Domain Crossing (CDC)

Lecturer: Po-Jen Chen

Graduate Institute of Electronics Engineering, National Taiwan University



NTU GIEE

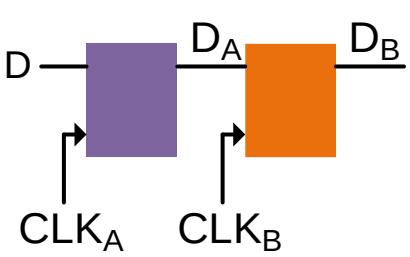


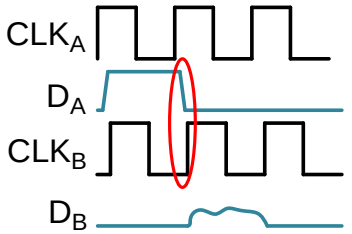
Clock Domain Crossing: IC Design Challenges

- Today's billion-gate ICs contain dozens of asynchronous clock domains
- There are multiple IP blocks, external interfaces, and variable-frequency power saving functions
- Conventional RTL simulation is not designed to verify metastability effects
- Static timing analysis does not address issues related to asynchronous clock domains
- As a result, many CDC-related bugs go undetected until the post-silicon testing stage
- CDC errors can break your IC!

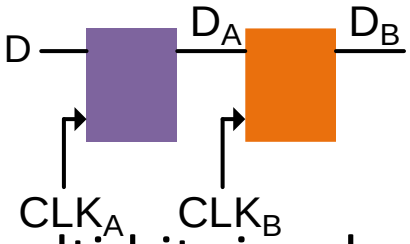
CDC Scenarios

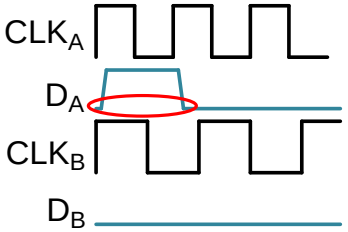
- Metastability in data transfer between asynchronous clock domains



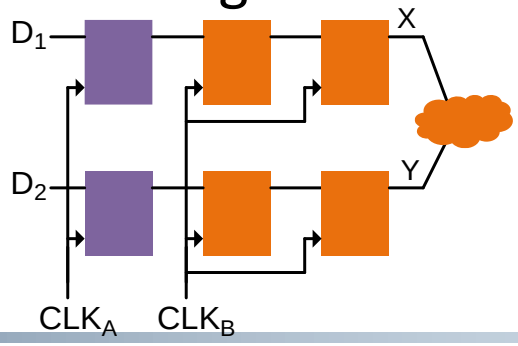


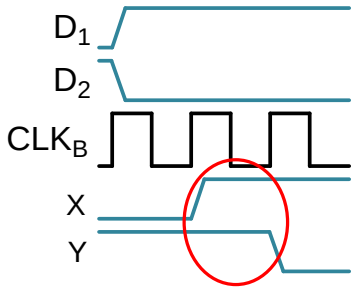
- Data loss in fast-to-slow clock transfers





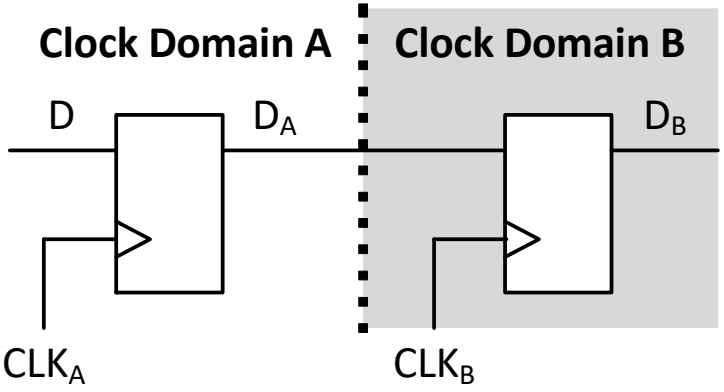
- Erroneous capture of multi-bit signals and re-convergence of synchronized signals

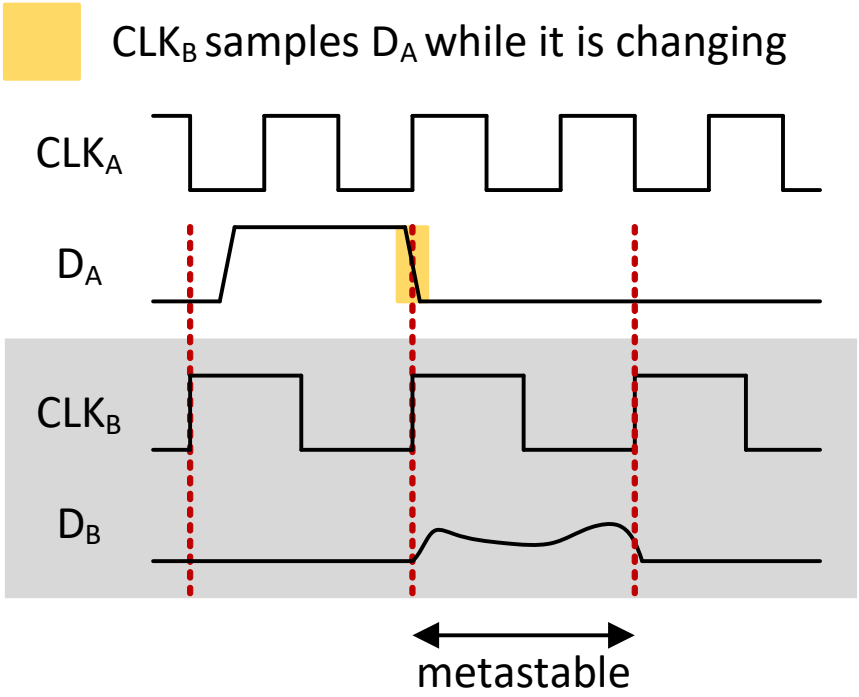




Metastability

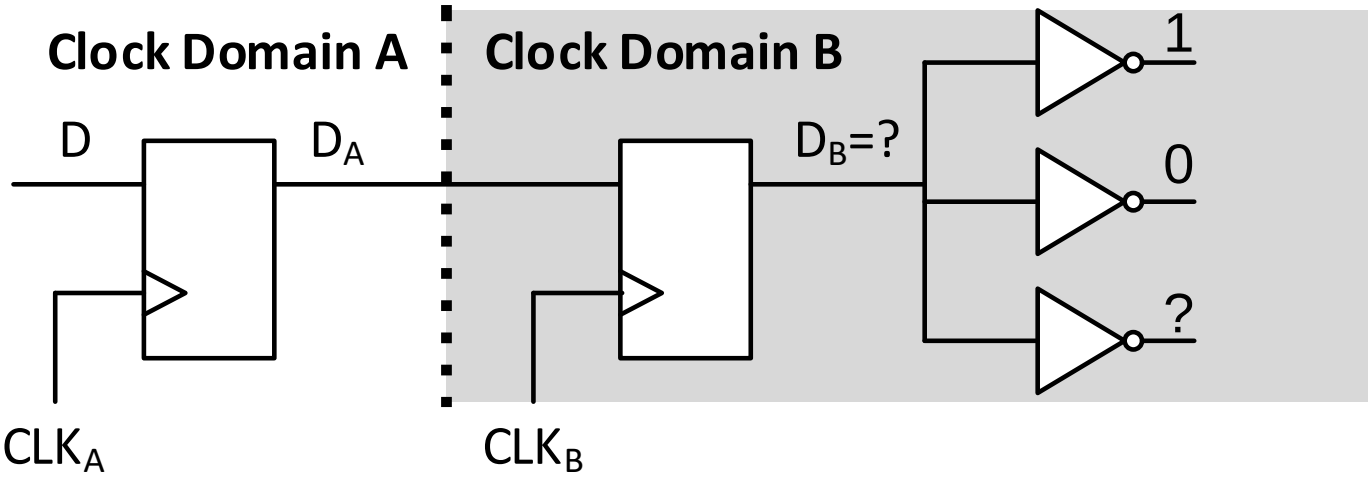
- Refers to signals that do not assume stable 0 or 1 states





Metastability Problem

- Since the metastable signal can fluctuate for some period of time, the other logics in the receiving clock domain may propagate incorrect signals





Mean Time Before Failure (MTBF)

- The signal does not settle to a known value after one clock cycle
- The average time between these failures is called MTBF

$$MTBF = e^{tr/\tau}/T_0 fa$$

tr: Transition time
τ: Time constant
T_0: Trigger parameter
f: Clock frequency
a = Asynchronous event trigger frequency

- A larger MTBF indicates a lower probability of capturing failures in the system



MTBF of A Flip-Flop

- For a typical flip-flop in an ASIC library with a 0.25µm process, we take the following parameters

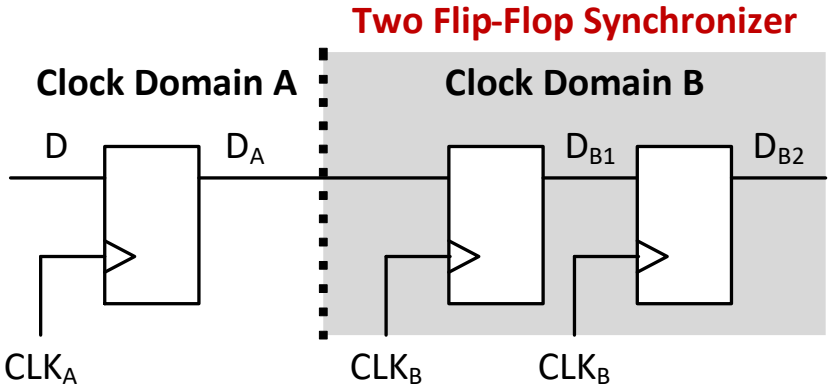
$$t_r = 2.3 \text{ ns}, \tau = 0.31 \text{ ns}, T_0 = 9.6 \text{ as}, f = 100 \text{ MHz}, a = 10 \text{ MHz}$$

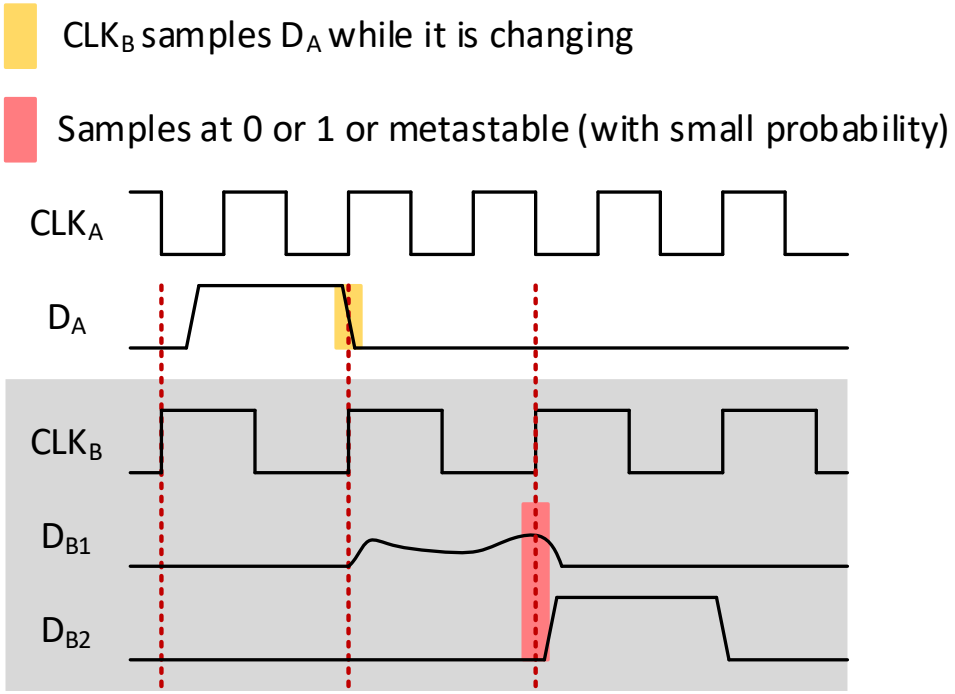
$$\text{MTBF} = 2.01 \text{ days}$$

- We assume that the flip-flop operates at 100 MHz and the asynchronous clock is also 100 MHz
- By continuously capturing its data at a frequency of 10 MHz, with changes occurring at each rising edge, it is possible to capture a metastable state approximately once every 2 days

Two Flip-Flop Synchronizer

- It generates synchronized signals





*For some high speed designs, the MTBF of a two flip-flop synchronizer is too short and a third flip-flop is added to increase the MTBF



MTBF of Two Flip-Flop Synchronizer

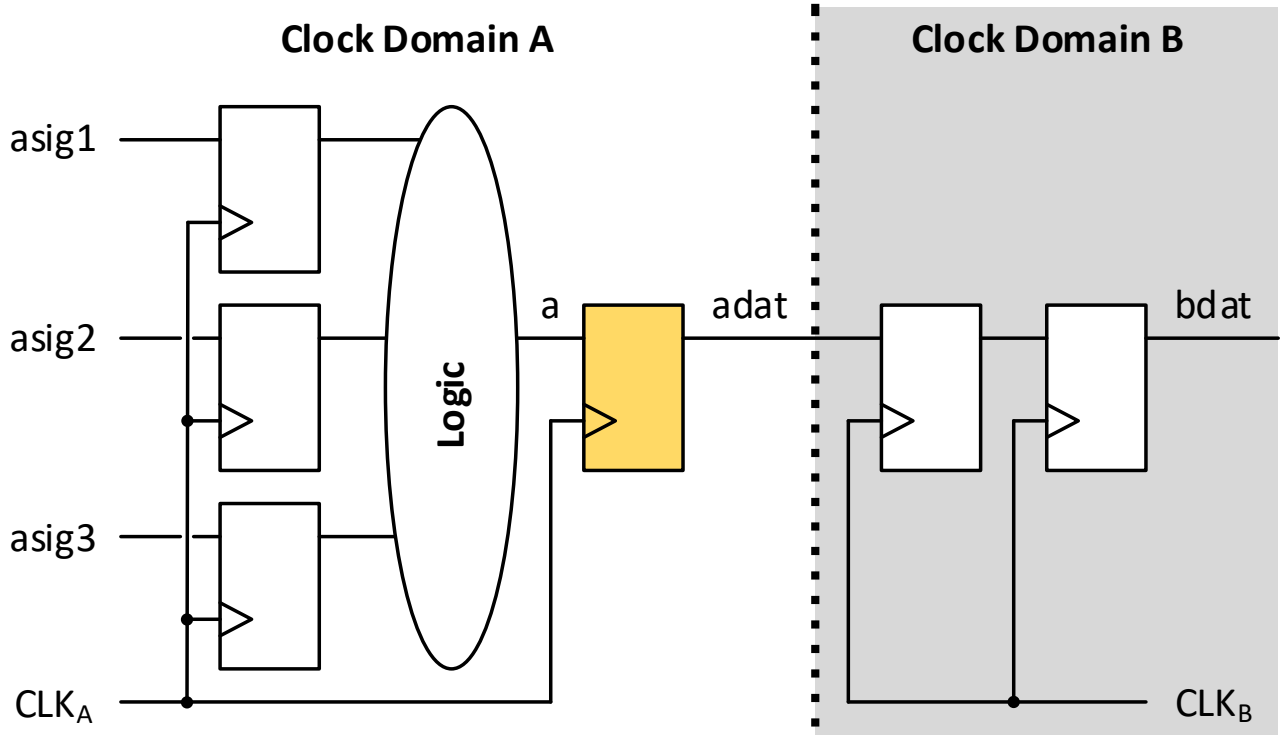
$$MTBF = \boxed{e^{(tr/\tau)}/T_0 fa} \times \boxed{e^{(tr/\tau)}/T_0 fa}$$

1st flip-flop 2nd flip-flop

- If we continue to use the parameters provided in the previous slide, the MTBF is 9.57×10^9 years
- From the results above, it can be seen that the 2 flip-flop method can eliminate the metastability issue

Synchronizing Signals from Sending Side

- The synchronization of signals from the sending side reduces the number of edges that can be captured on the receiving side, effectively increasing the MTBF





Single-Bit CDC



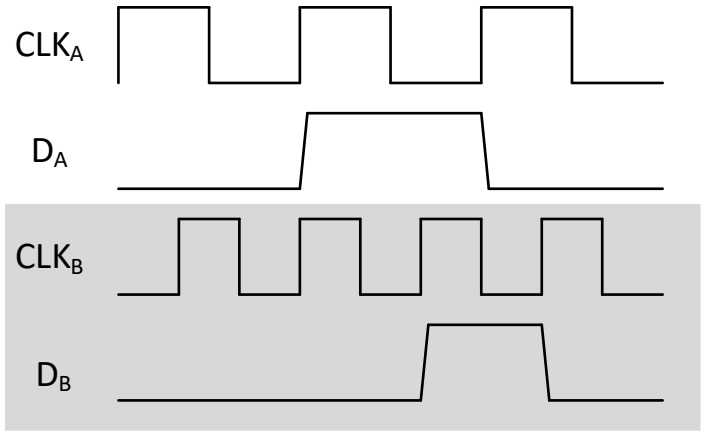
Reliable Signal Passing: "Three Edge"

- Synchronizing slower signals into a faster clock domain is generally not a problem ($f_{clk\text{-}fast} > 1.5 \times f_{clk\text{-}slow}$)
- Input data values must be stable for three destination clock edges, including both positive and negative edges
- The "three edge" guideline is the safest design condition
- CDC solutions include both open-loop and closed-loop approaches
- Implementations of the closed-loop solution automatically ensure that at least three edges are detected for all CDC signals



Slow-to-Fast Clock Transfer

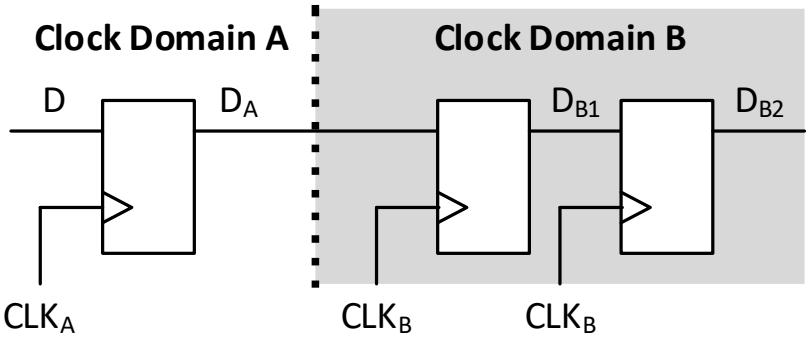
- $f_{clk\text{-}fast} > 1.5 \times f_{clk\text{-}slow}$
- It is safe to transfer data from slow to fast clock domain



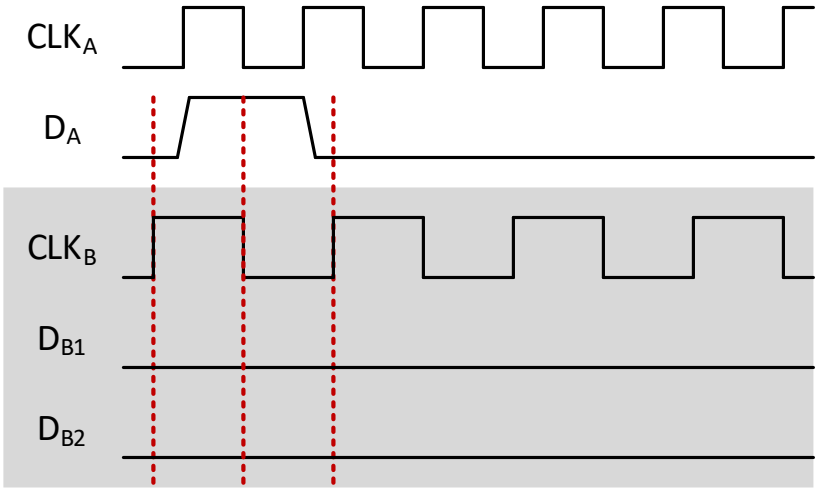
- The “three edge” in the receiving clock domain guarantees that at least one positive edge occurs
  - Positive, negative, positive
  - Negative, positive, negative



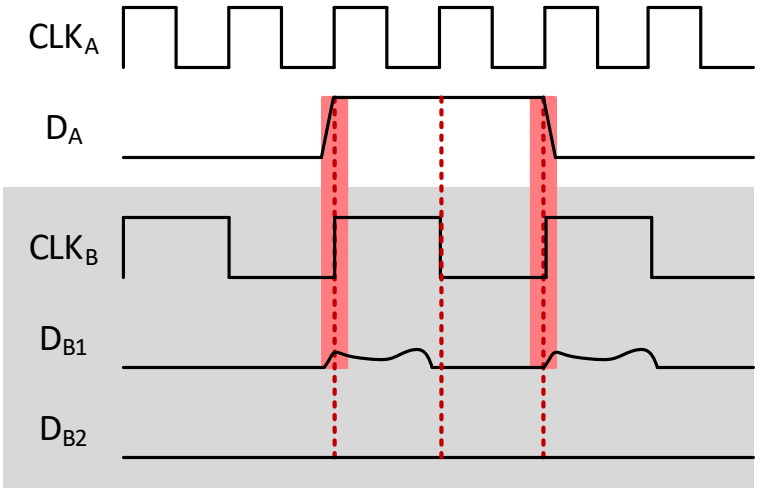
Fail to Meet "Three Edge" Requirement



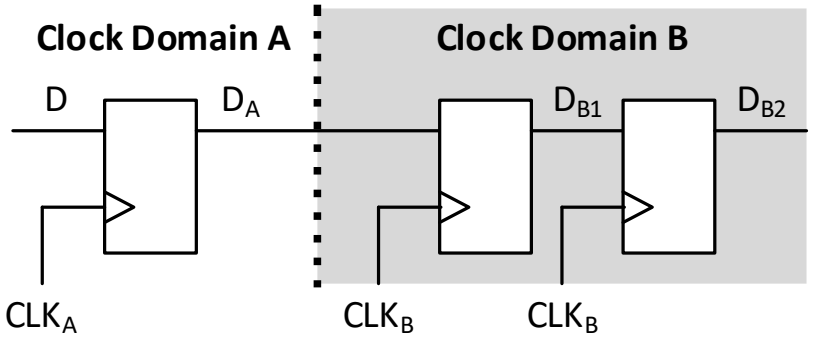
D_A: Too short



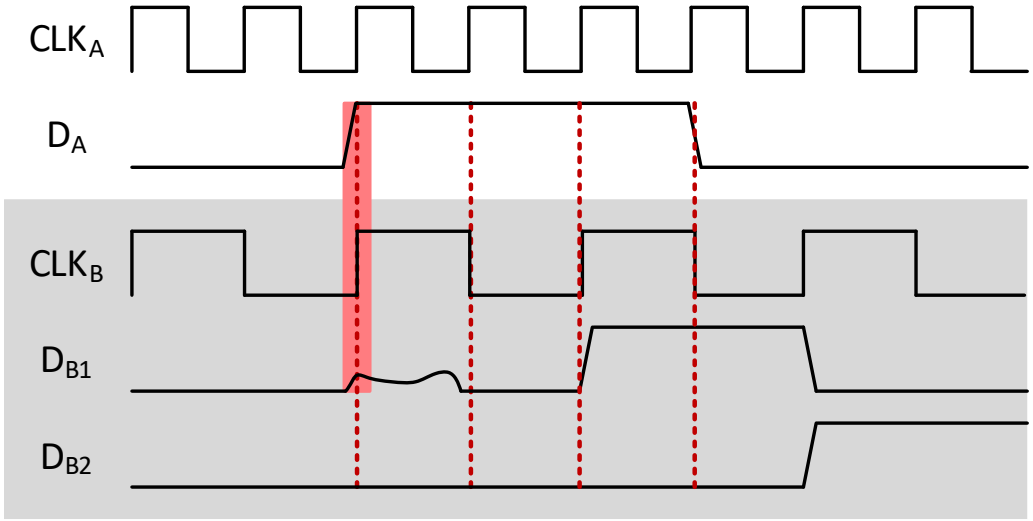
D_A: Not long enough



Success to Meet "Three Edge" Requirement



D_A: Long enough

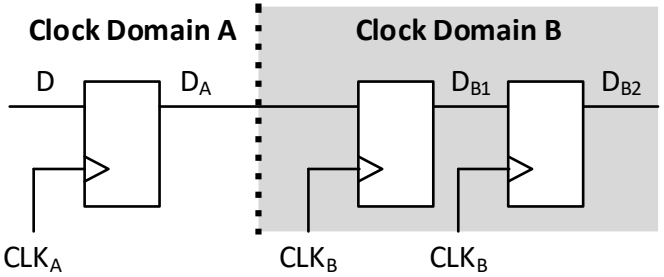


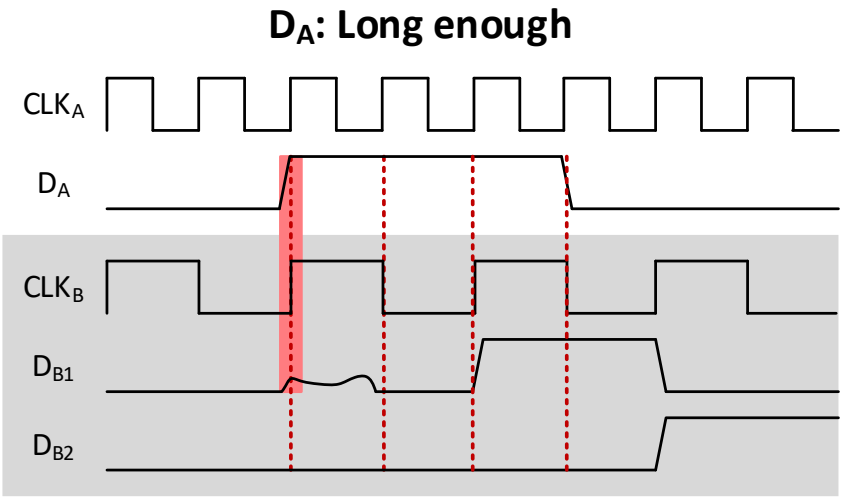
$$T_{D\text{-fast}} > 1.5 \times T_{period\text{-slow}}$$



Open-Loop Solution

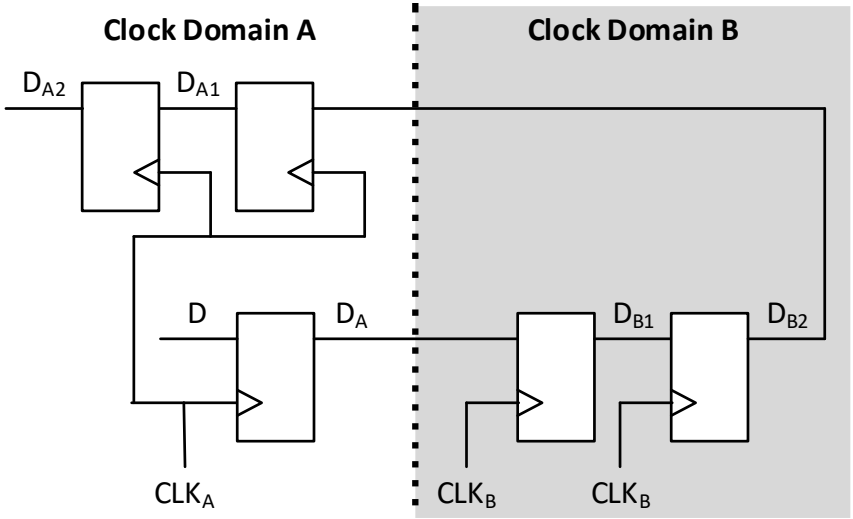
- It is the fastest way to pass signals across CDC boundaries
- It is recommended to add a SystemVerilog assertion to detect whether the signals satisfy the three-edge requirement

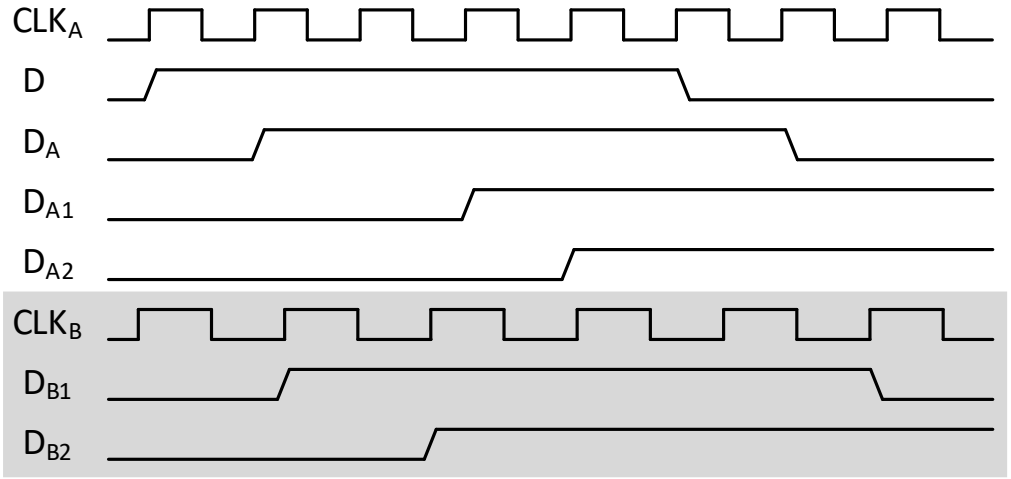




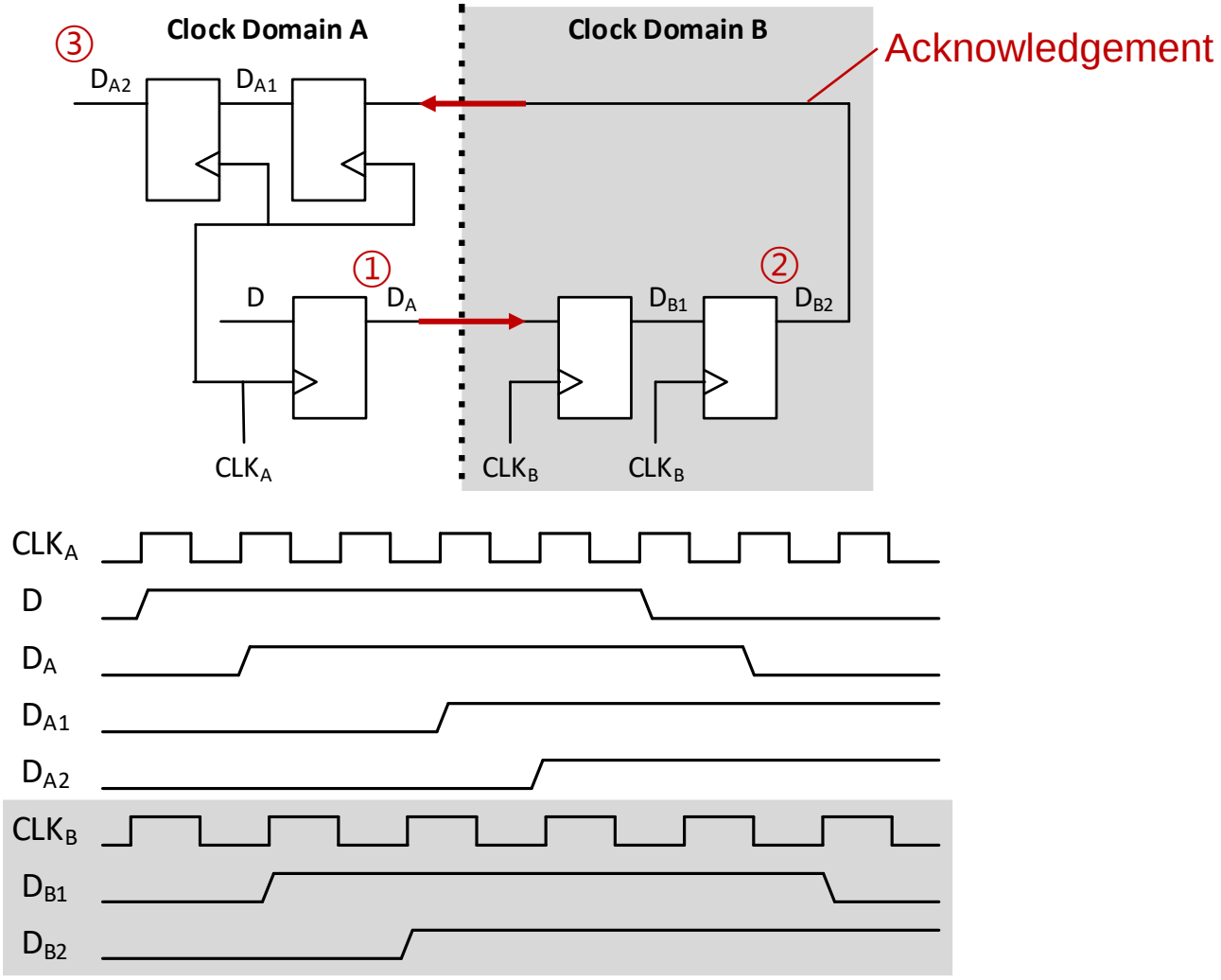


Closed-Loop Solution



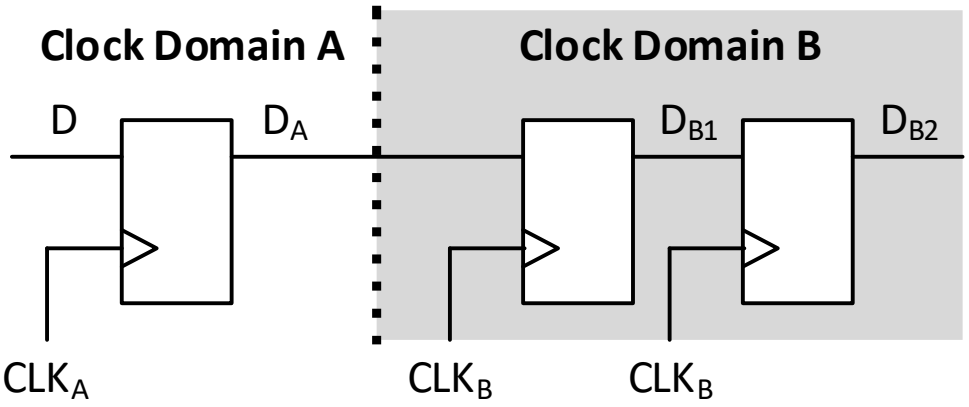


Closed-Loop Solution





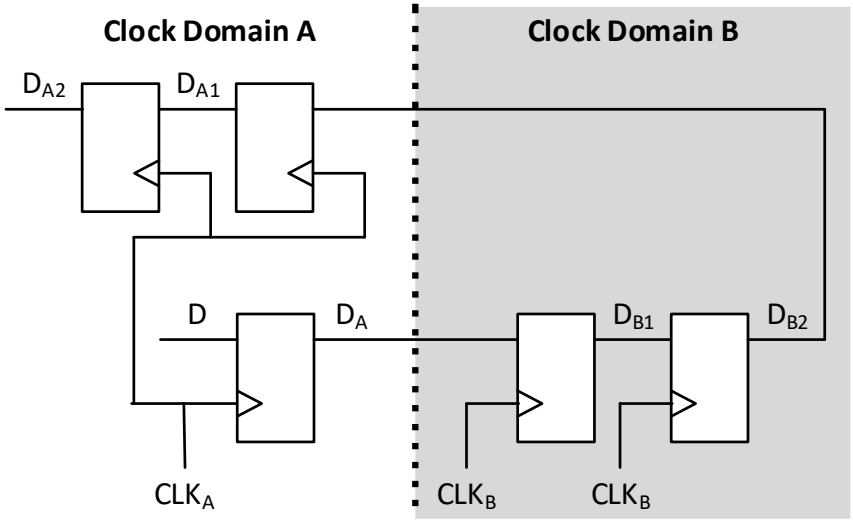
Pros and Cons of Open-Loop Solution



- Pros: The fastest way to pass signals across boundaries without requiring acknowledgment of the received signal
- Cons: If the relative clock frequency is not fixed, the three-edge requirement cannot be guaranteed



Pros and Cons of Closed-Loop Solution



- Pros: A safe technique to acknowledge that the signal is recognized and captured into the next clock domain
- Cons: There exists a potentially considerable delay associated with synchronizing signals in both directions before allowing the sent signal to change



Multi-Bit CDC



Multi-Bit CDC

- Synchronizers do not guarantee the safe delivery of data when passing multi-bit signals between clock domains
- Multi-bit signals that are synchronized to a single clock may experience small skews when data changes
  - Occasionally cause the signals to be captured on different rising clock edges in a second clock domain
- Multi-bit CDC strategies must be employed to avoid skewed capturing of the multi-bit value



Strategies for Multi-Bit CDC

- (1) Multi-bit signal consolidation
  - Consolidate a multi-bit signal into a single-bit signal
- (2) Multi-cycle path
  - Use a synchronized load signal to transfer multi-bit signals
- (3) FIFO (First In, First Out)
  - Use registers or memory to store data
  - Use Gray code for the read/write pointers



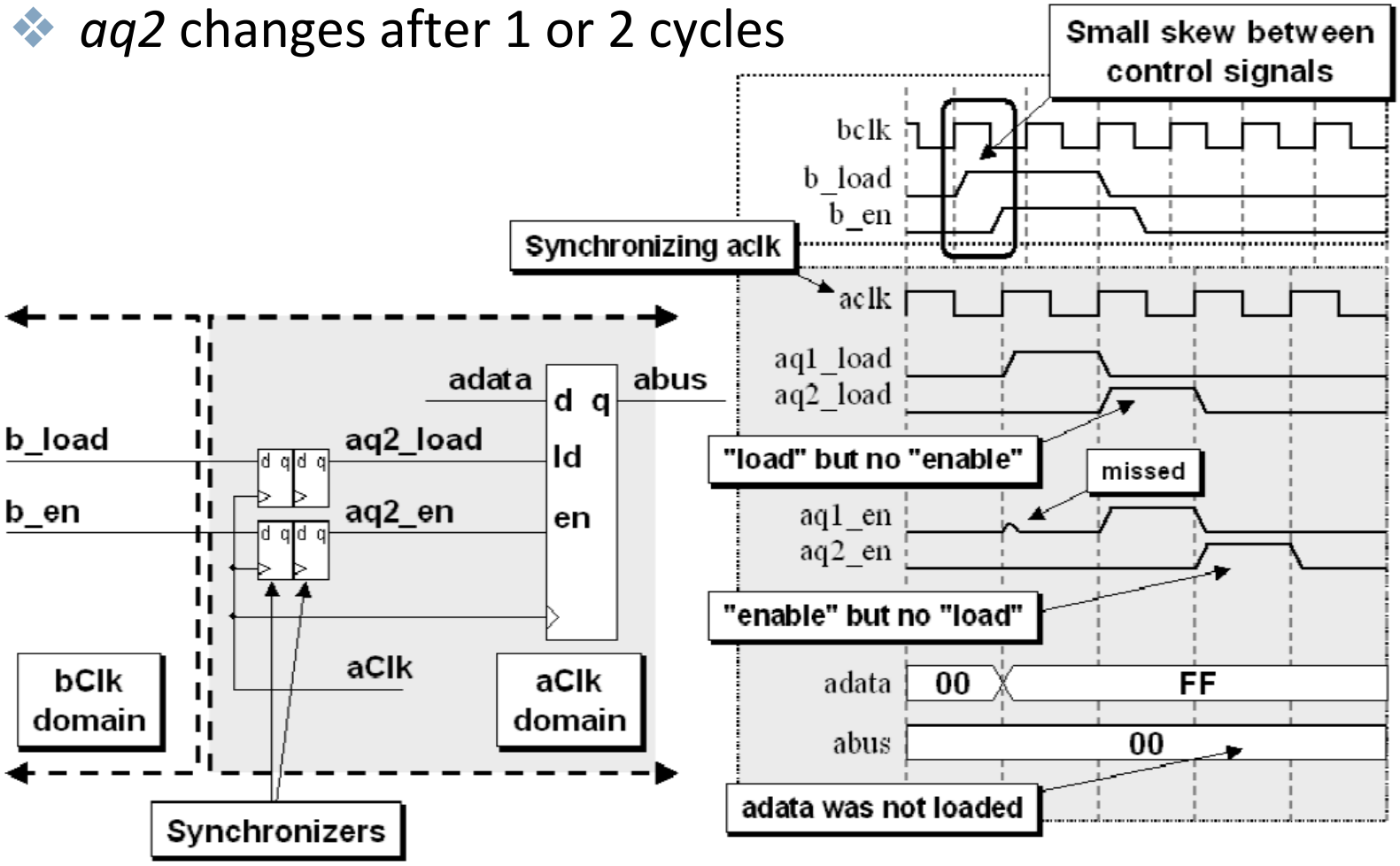
Multi-bit Signal Consolidation

- Skew may occur between CDC signals, preventing the data from being properly received by the receiving side

- If possible, consolidate multiple CDC signals into a single 1-bit CDC signal

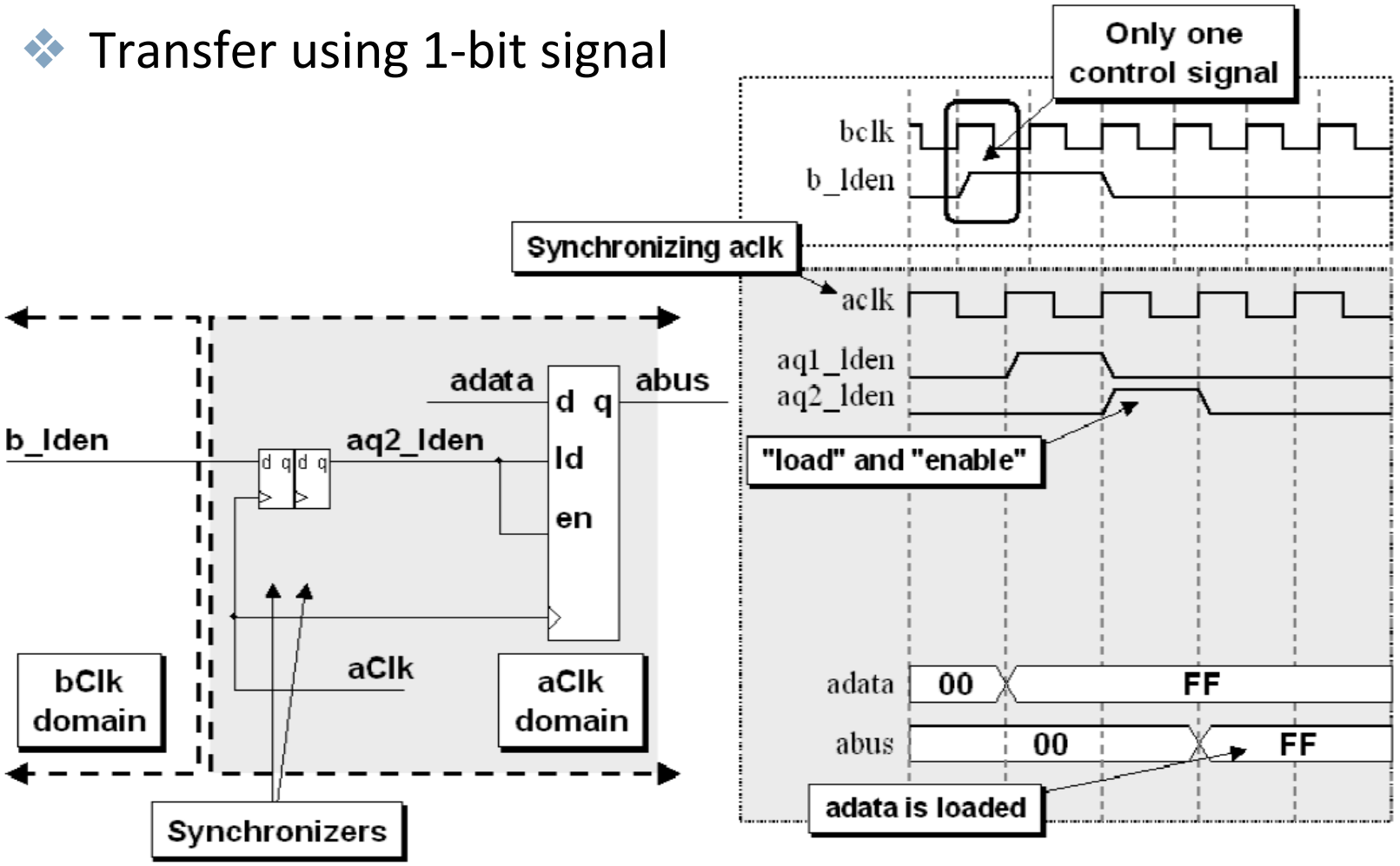
Problem 1 - Two Simultaneously Signals

- *aq2* changes after 1 or 2 cycles



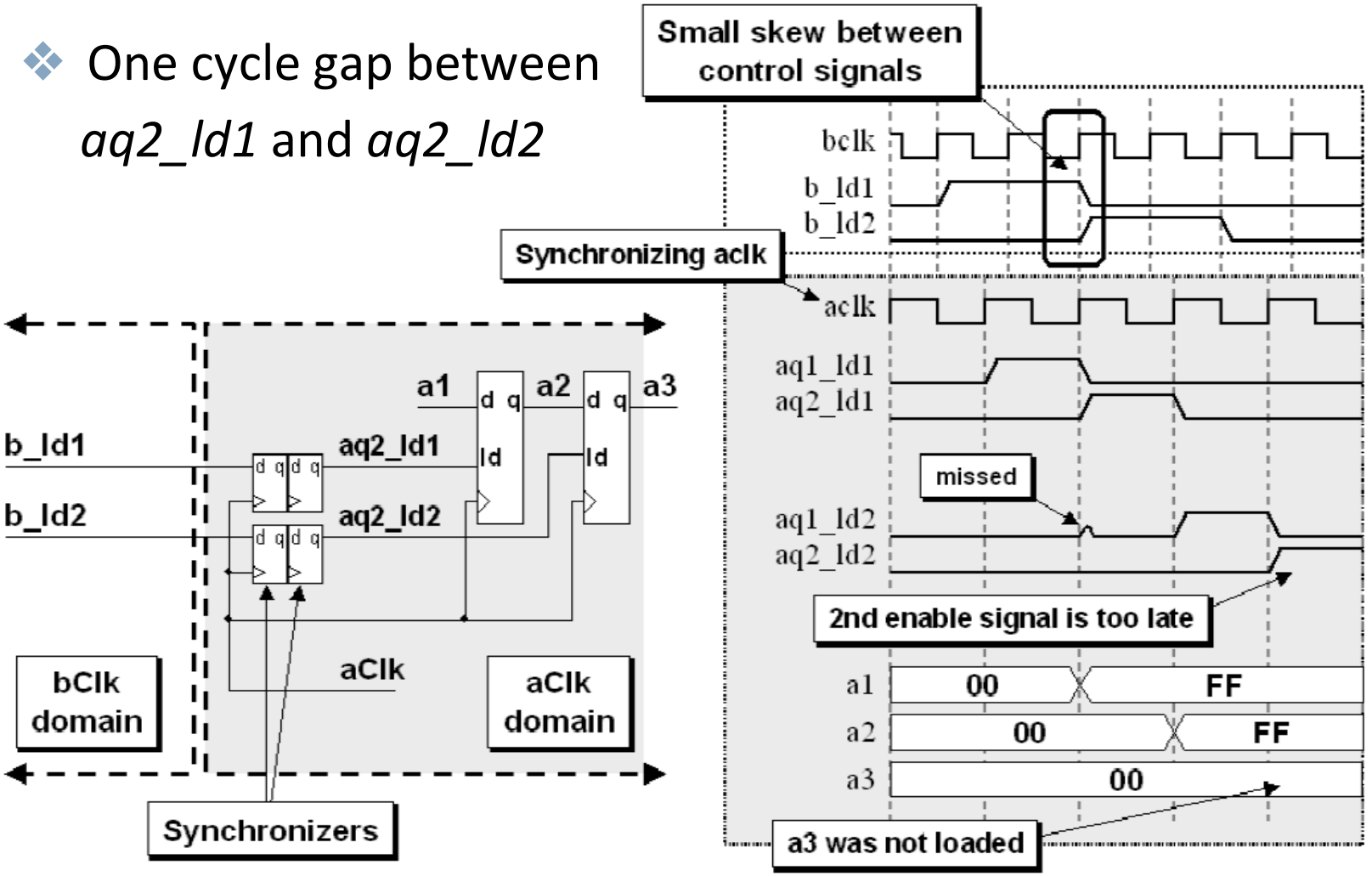
Solution 1 – Consolidating the Signals

- Transfer using 1-bit signal



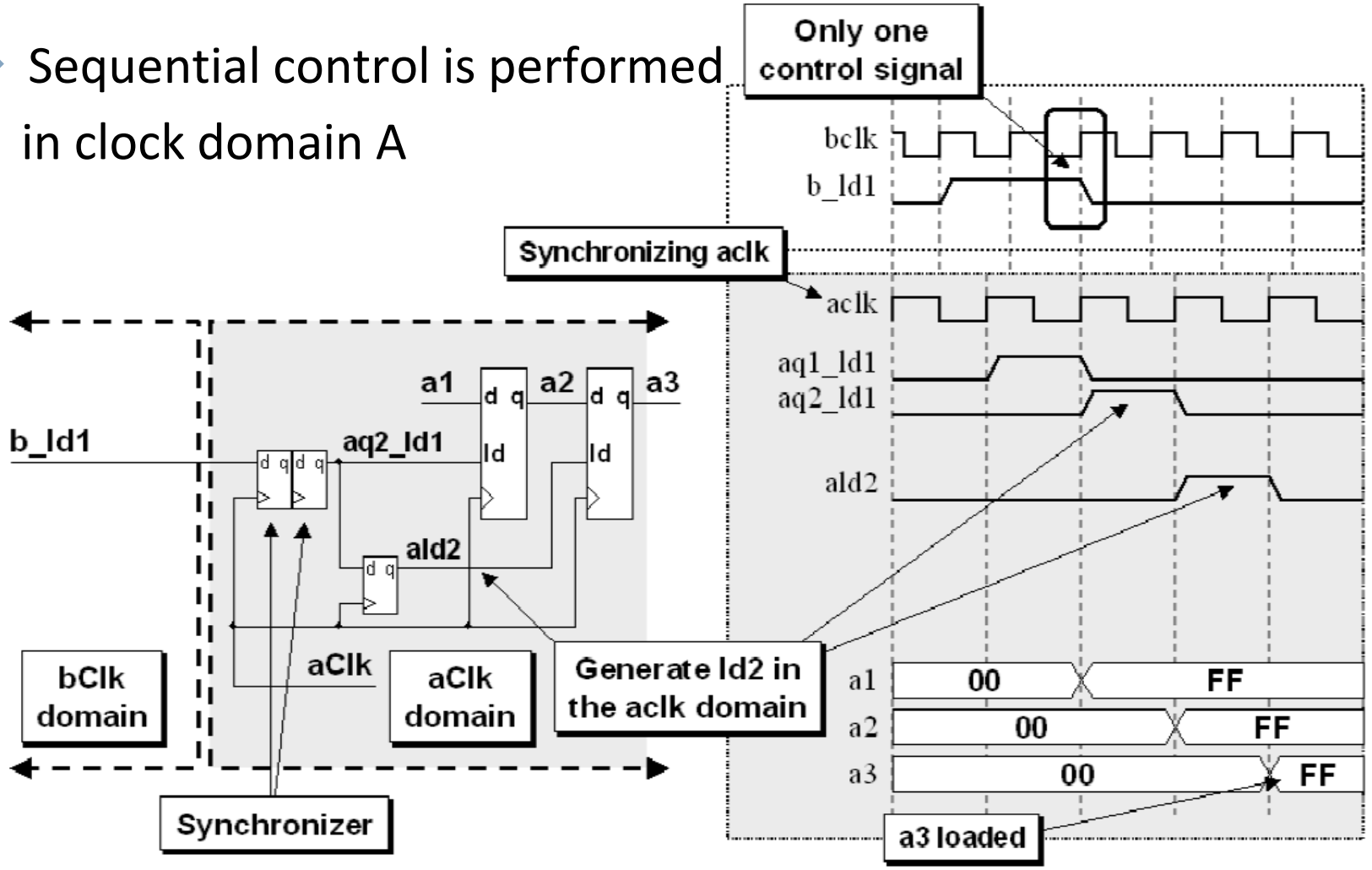
Problem 2 - Two Sequencing Signals

- One cycle gap between *aq2_ld1* and *aq2_ld2*



Solution 2 – Consolidation & 1 Flip-Flop

- Sequential control is performed in clock domain A



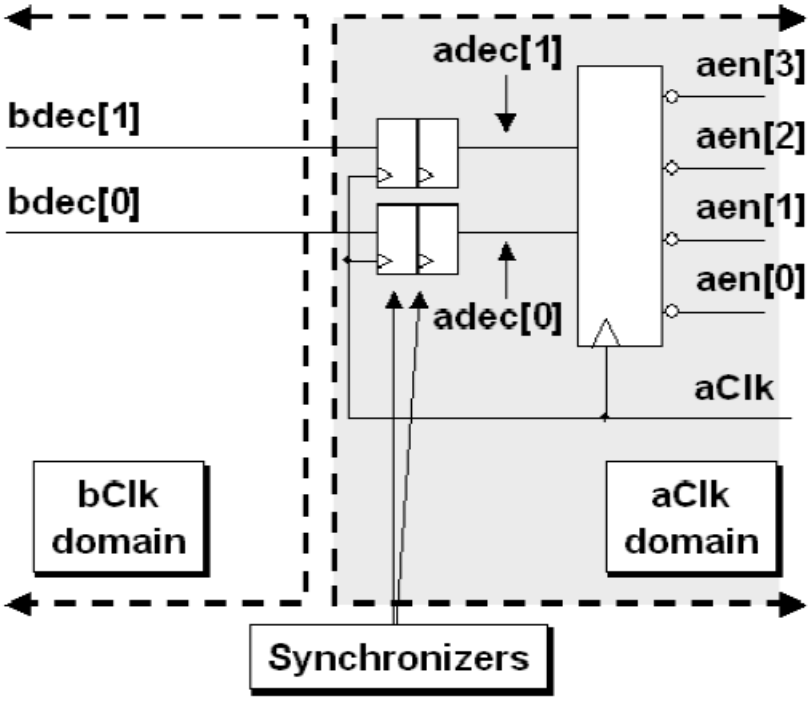


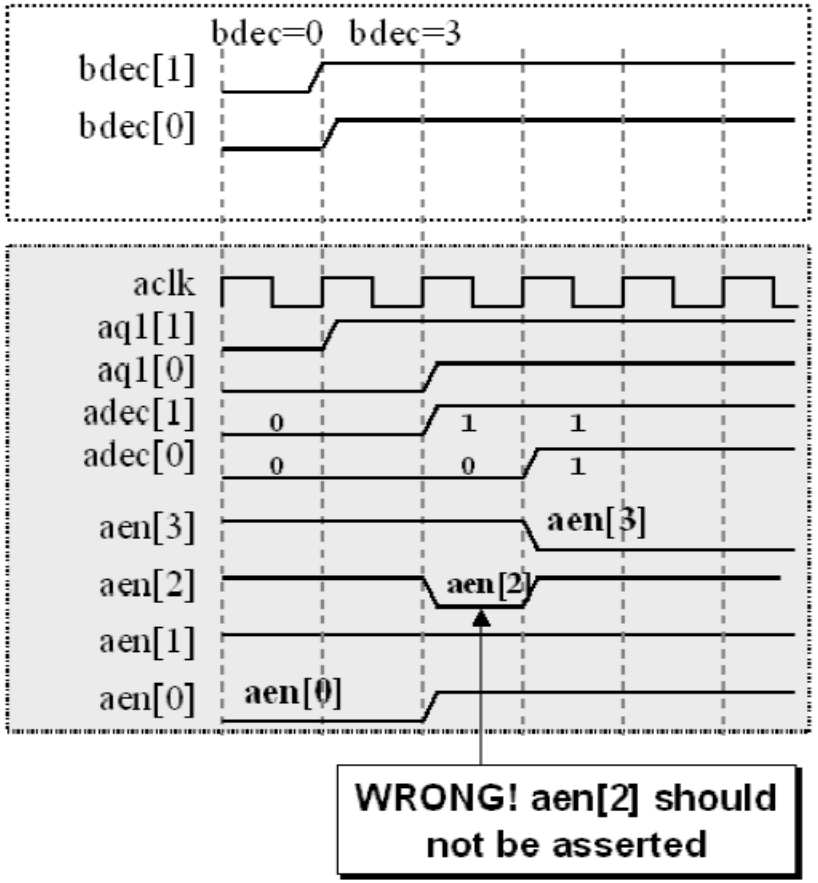
Multi-Cycle Path (MCP)

- An MCP formulation refers to sending unsynchronized data to a receiving clock domain, paired with a synchronized control signal
- This strategy transmits multiple CDC signals without synchronization while simultaneously passing a synchronized enable signal to the receiving clock domain
- The receiving clock domain is not allowed to capture the multi-bit signals until the synchronized enable signal has arrived at the receiving register

Problem – Multiple CDC Signals

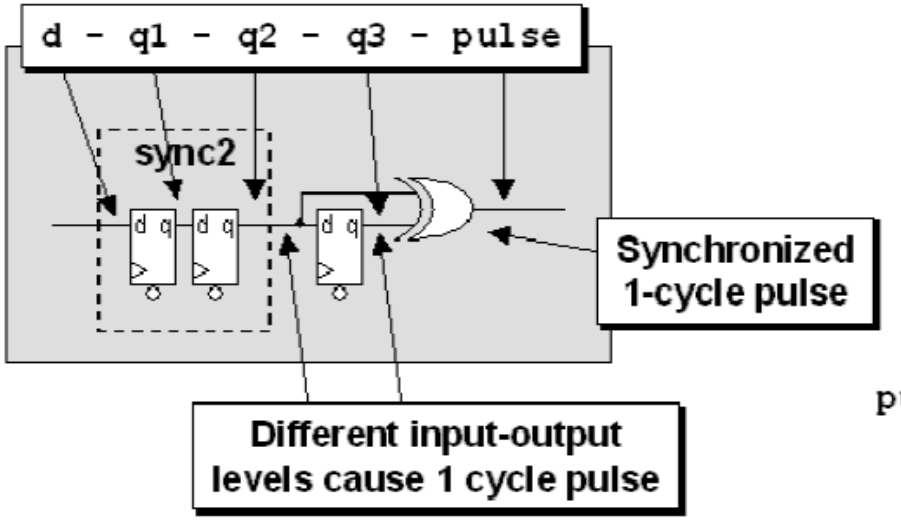
- One-hot encoder: 00 -> 1110, 01-> 1101, 10 ->1011, 11 -> 0111

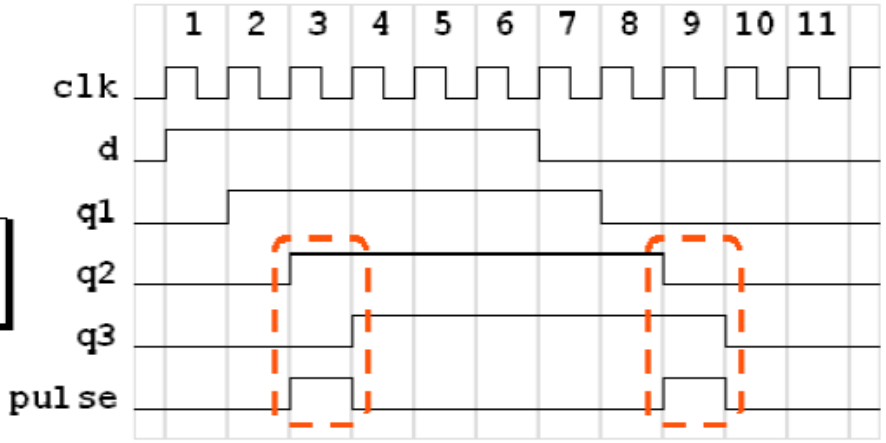




Synchronized Pulse Generation Logic (SPGL)

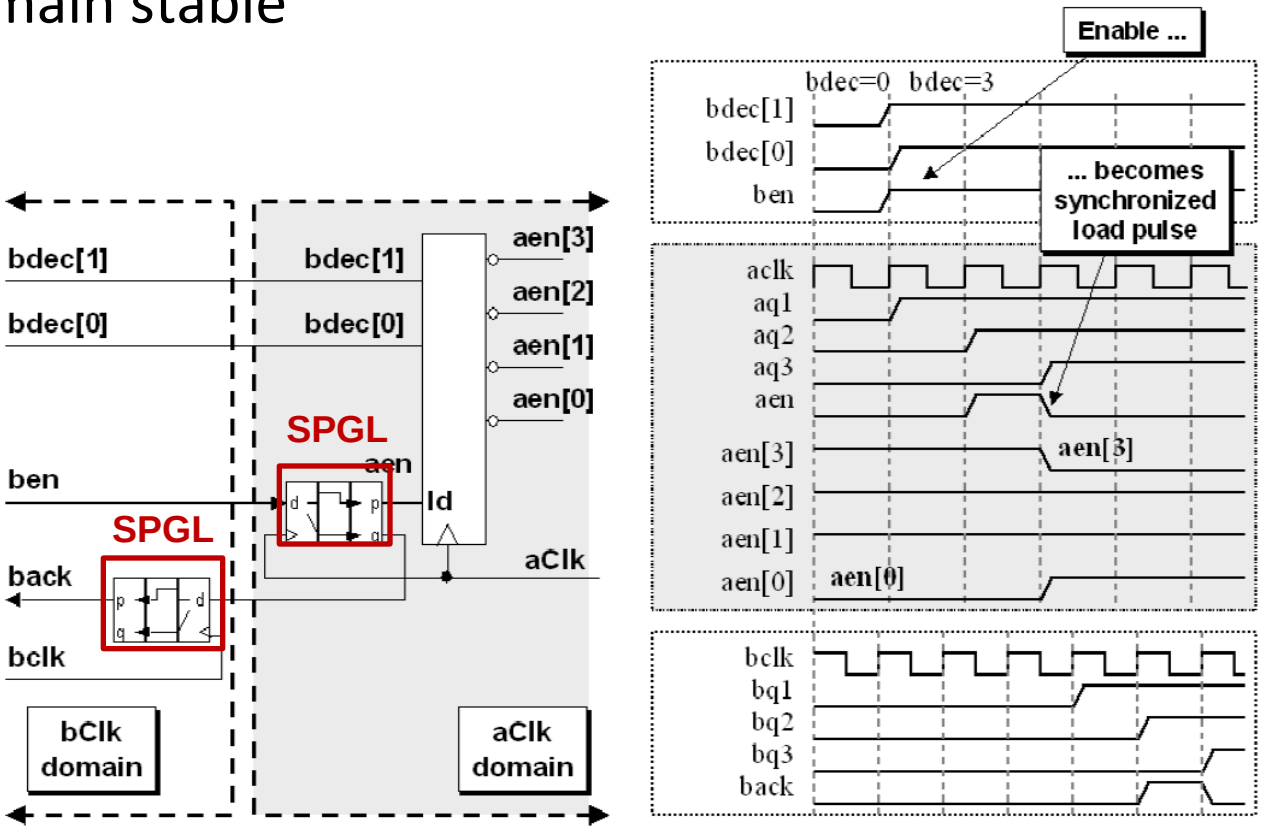
- It consists of a two flip-flop synchronizer and a pulse generator
- Each toggle of *d* generates a one-cycle pulse





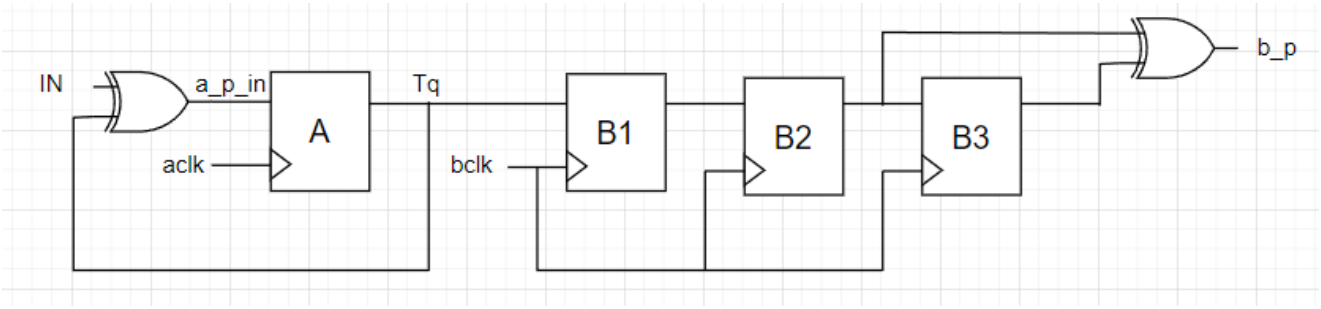
Solution – Synchronized Enable Pulse

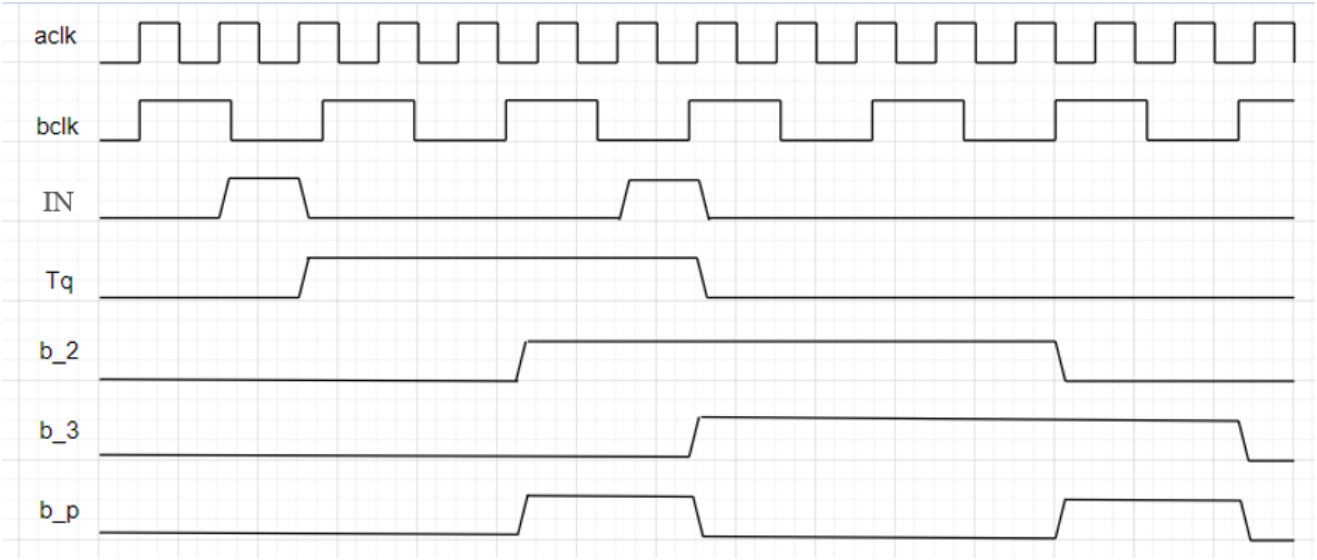
- The unsynchronized data is held for multiple cycles before being captured, ensuring that the data values remain stable



Pulse Synchronizer

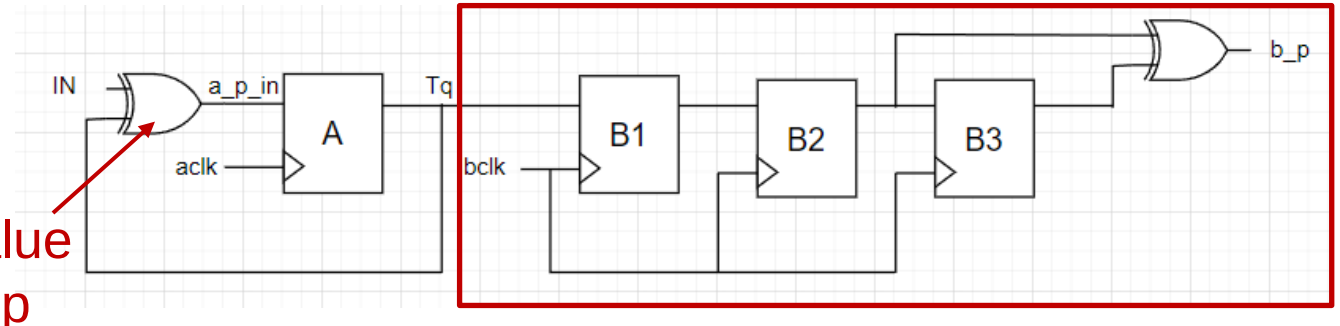
- Transfer one-cycle pulse from sending to receiving domain

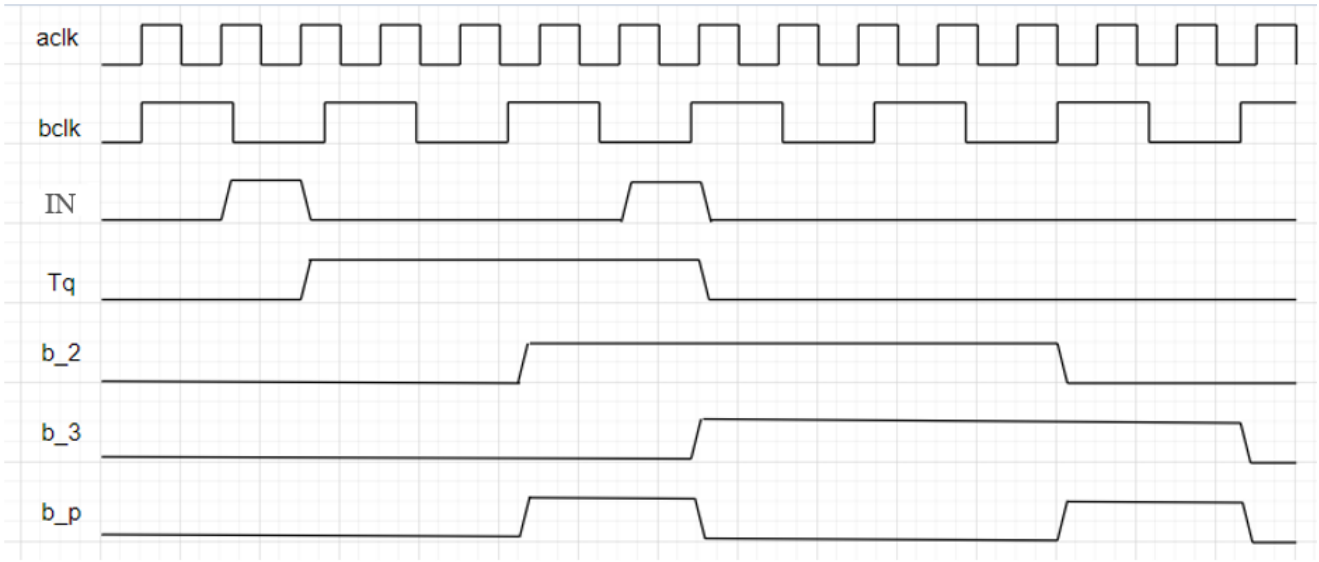




Pulse Synchronizer

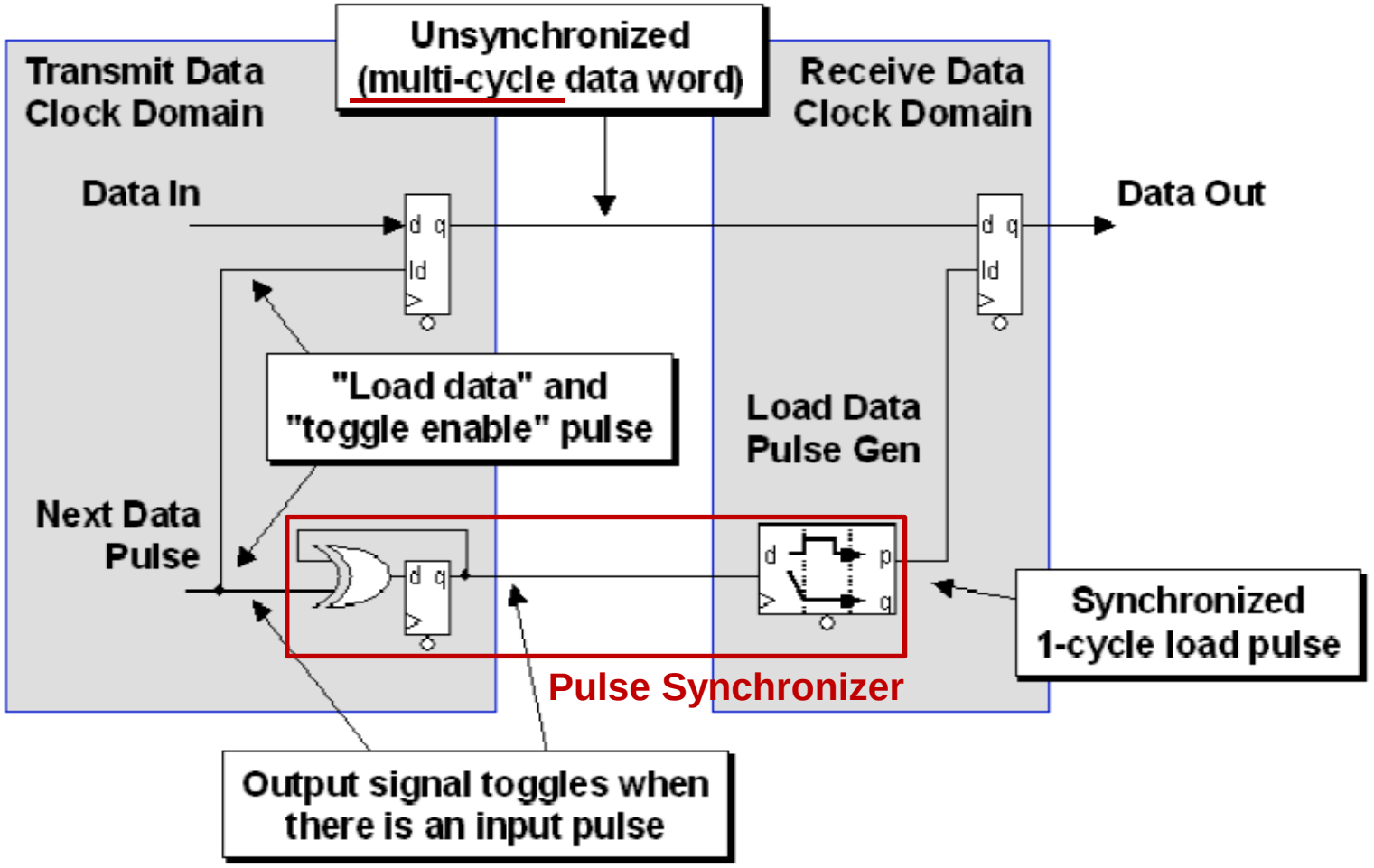
- Transfer one-cycle pulse from sending to receiving domain



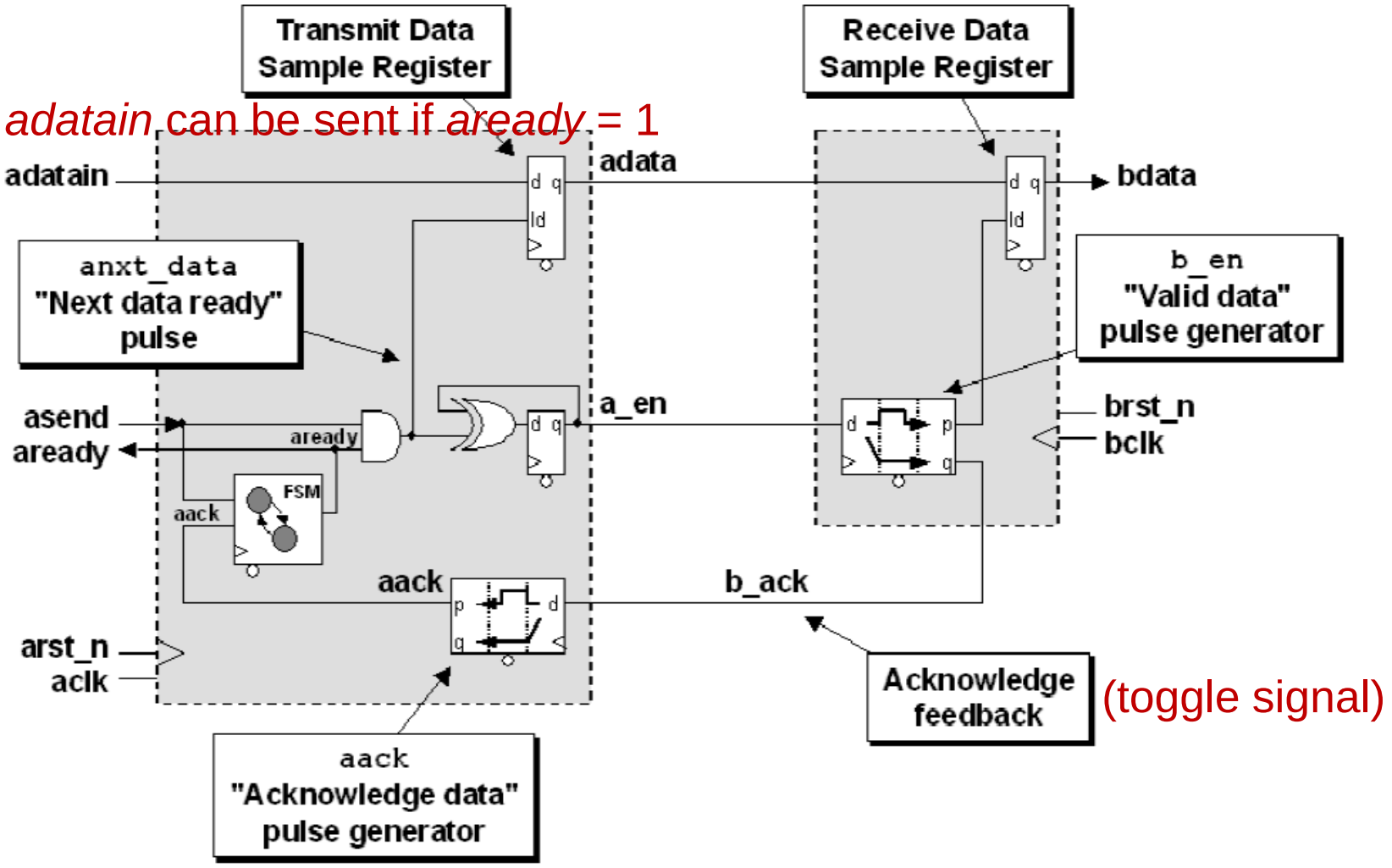




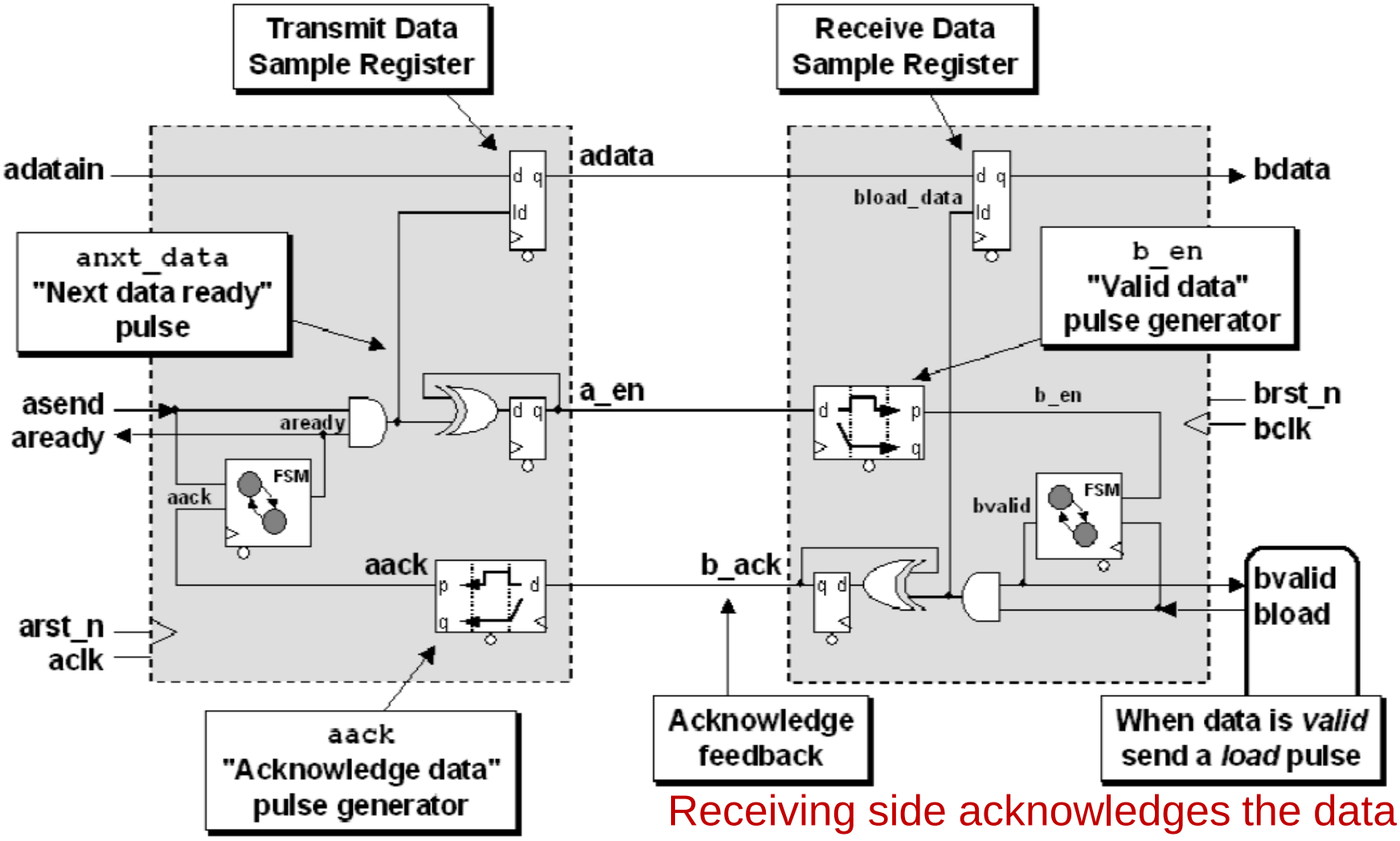
MCP Toggle-Pulse Generation Design



MCP Formulation with Feedback



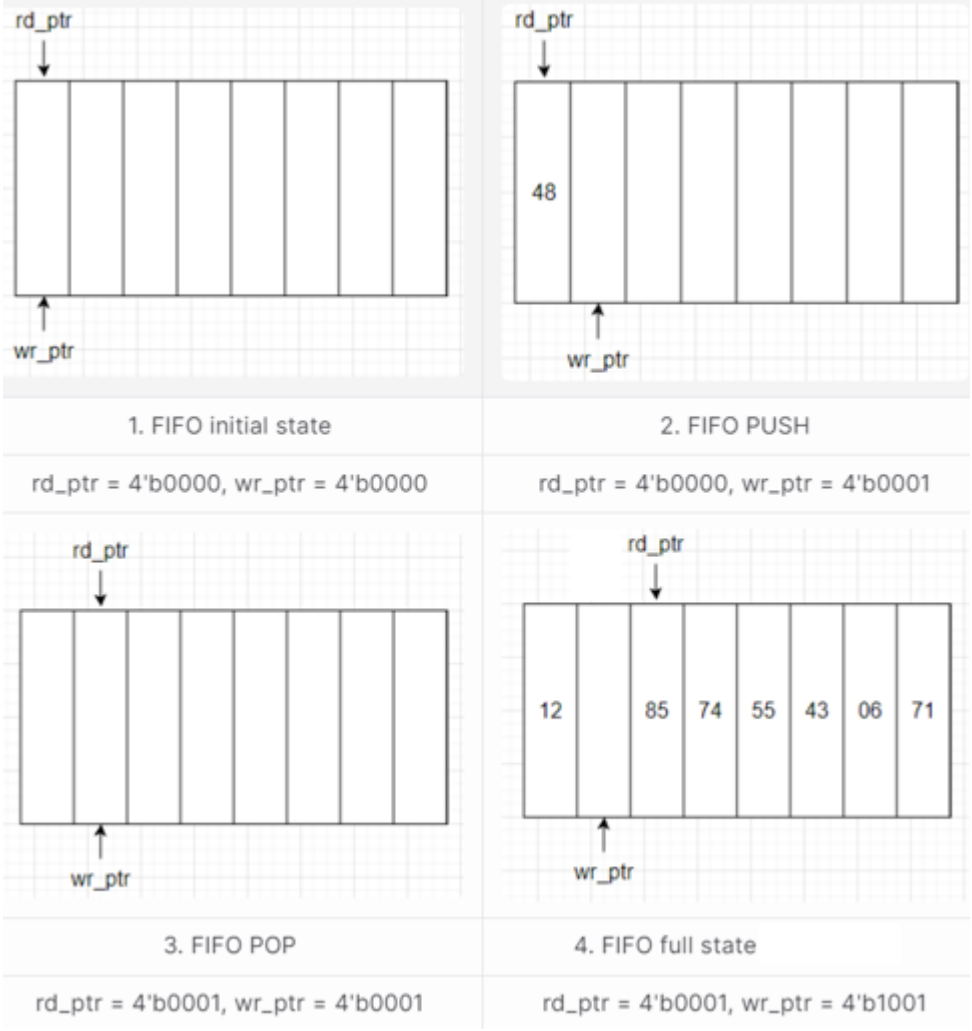
MCP Formulation with Acknowledge Feedback



Receiving side acknowledges the data if it has received the data and *bload* = 1

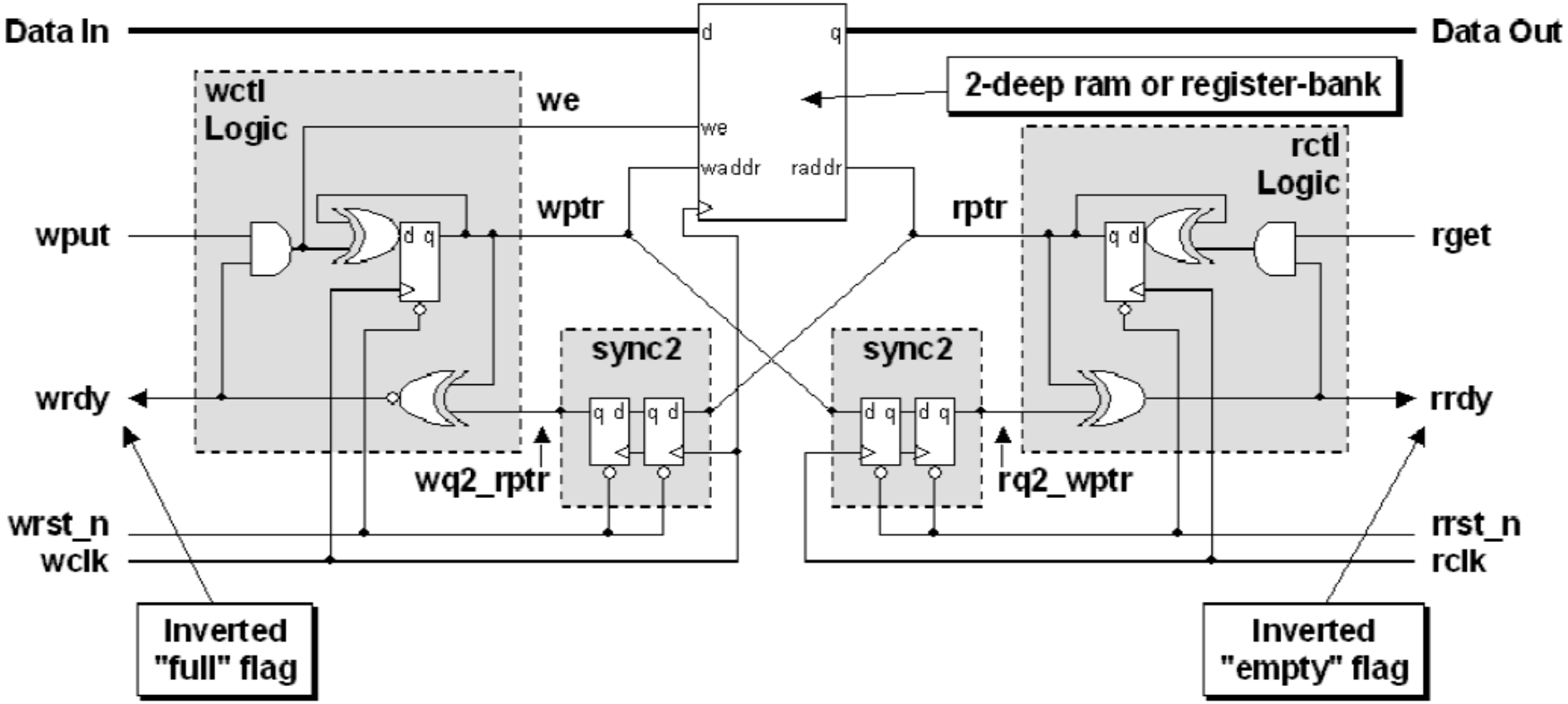
FIFO

- For a FIFO with 8 entries, rd_ptr/wr_ptr are 4 bits
- Data is inserted from the write clock domain
- Data is removed from the read clock domain



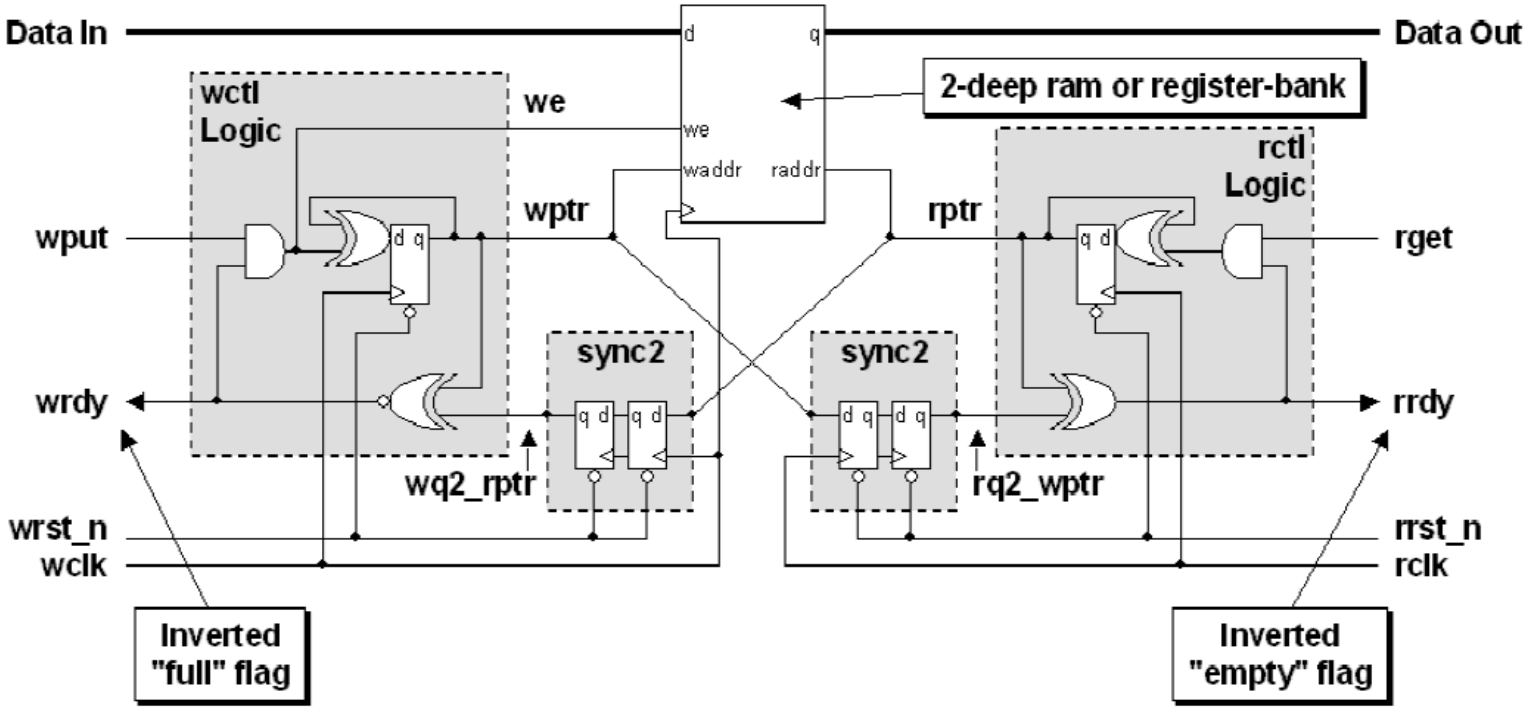
2-Register FIFO

- Use gray code (1-bit) for pointers
- Two flip-flop synchronizer



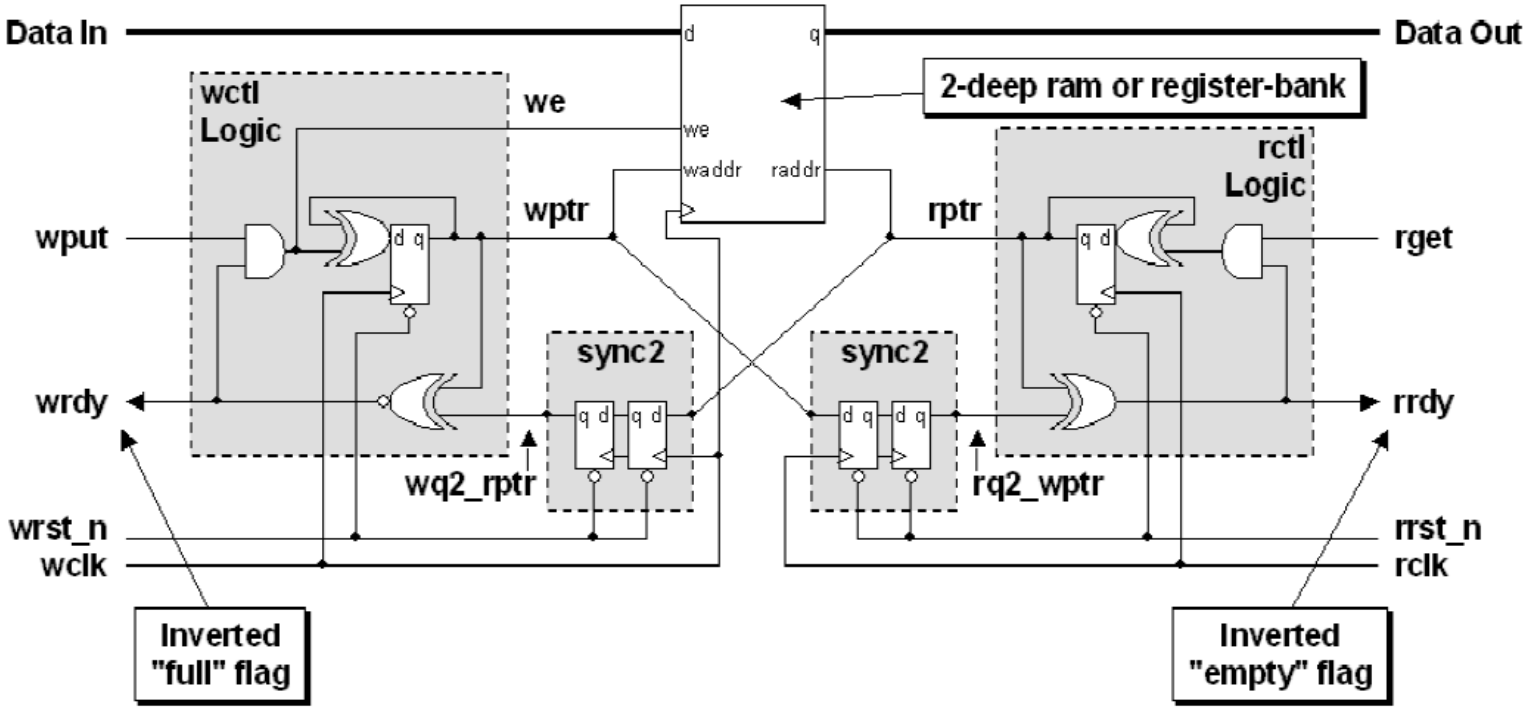
2-Register FIFO

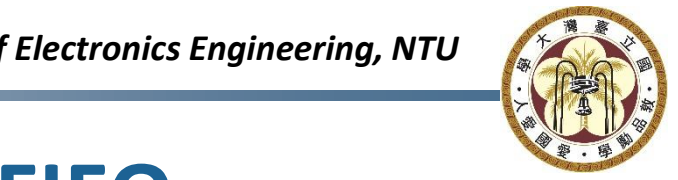
- Initial state: *wptr* = 0, *wrdy* = 1, *rptr* = 0, *rrdy* = 0
- 1. *wput* = 1, *data_in* changes
- 2. *data_in* is stored, *wptr* = 1, *wrdy* = 0, *rrdy* = 1



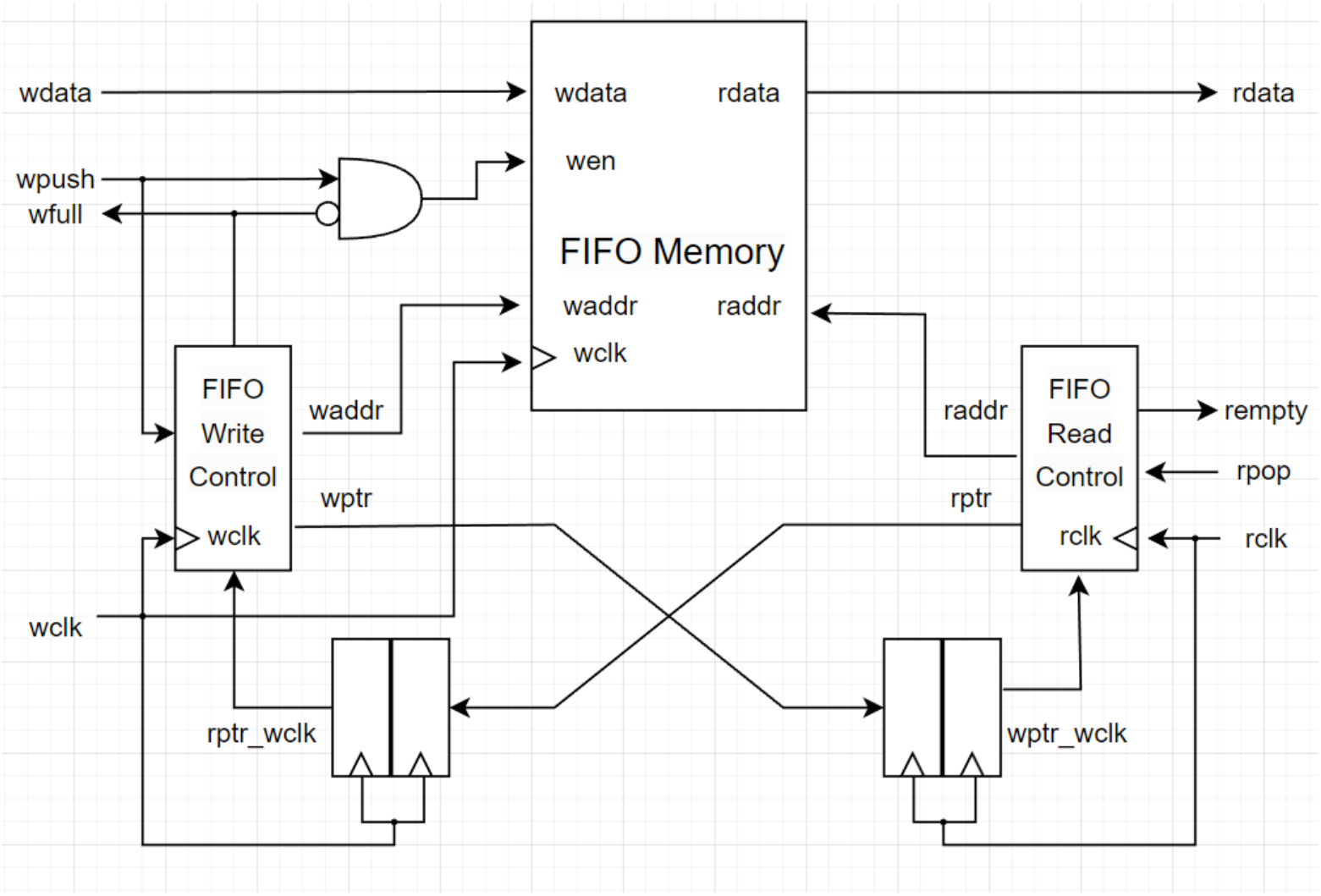
2-Register FIFO

- 2. *data_in* is stored, *wptr* = 1, *wrdy* = 0, *rrdy* = 1
- 3. read *data_out*, *rget* = 1
- 4. *rptr* = 1, *rrdy* = 0, *wrdy* = 1





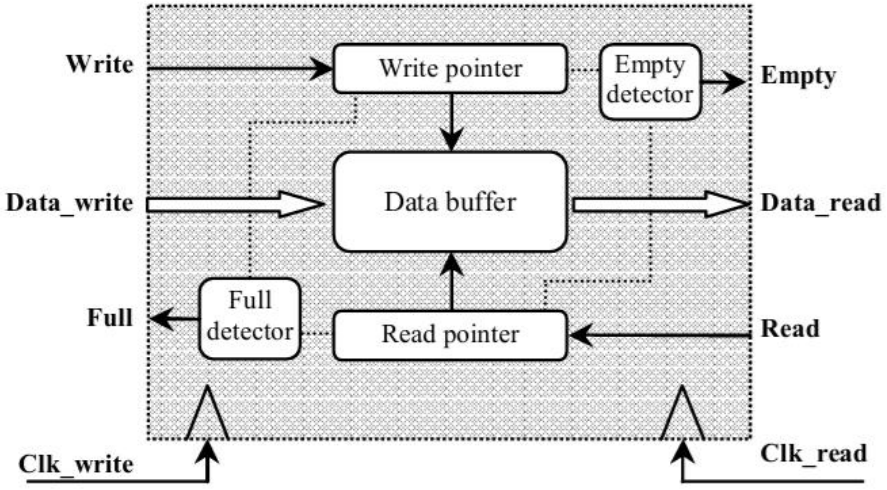
Asynchronous FIFO





Asynchronous FIFO

- N-register FIFO can store at most N-1 data elements
- Address controller generates write pointer or read pointer
- Empty: write pointer = read pointer
- Full: write pointer next = read pointer



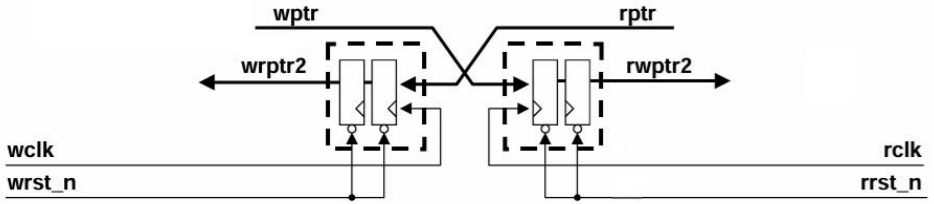
	Signal	Description
Sender interface	Data_write	Data to be written into the FIFO
	Write	Input signal requesting a write into the FIFO
	Full	Output signal indicating the fullness of the FIFO
	Clk_write	Sender clock signal
Receiver interface	Data_read	Output data from the FIFO
	Read	Input signal requesting a read in the FIFO
	Empty	Output signal indicating the emptiness of the FIFO
	Clk_read	Receiver clock signal

Address Controller

- The address uses Gray code

Binary Counter	Gray Code Counter
00	00
01	01
10	11
11	10

- The read/write pointer is a CDC signal
  - Two flip-flop synchronizer is needed
  - Gray code is used to avoid multi-bit CDC



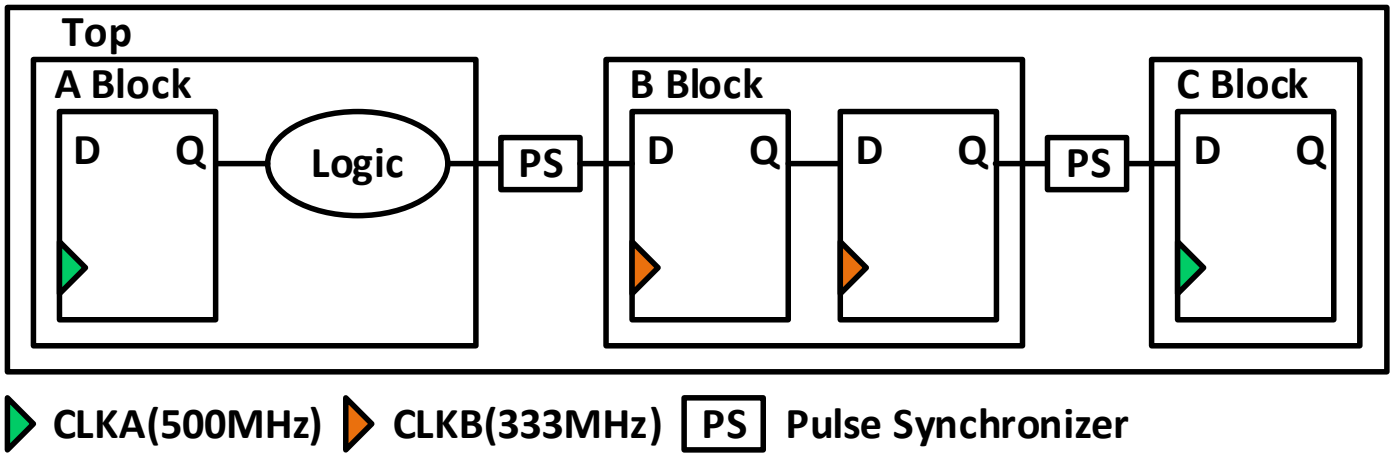


Synthesis Constraints



False Path: Synthesis with Asynchronous Clocks

- Disable timing-based checks on any path that crosses an asynchronous boundary



Method 1

```
set_false_path -from [get_clocks CLKA] -to [get_clocks CLKB]
set_false_path -from [get_clocks CLKB] -to [get_clocks CLKA]
```

Method 2

```
set_clock_groups -asynchronous -group {CLKA} -group {CLKB}
```



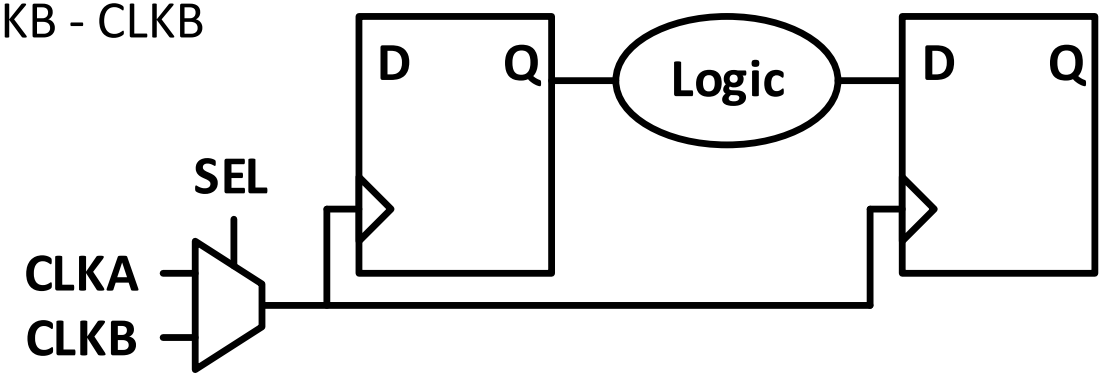
Synthesis with Asynchronous Clocks

- set_clock_groups -asynchronous -group {CLKA CLKB} -group {CLKC CLKD}
  - No timing checks from CLKA to CLKC or CLKD
  - No timing checks from CLKB to CLKC or CLKD
  - No timing checks from CLKC to CLKA or CLKB
  - No timing checks from CLKD to CLKA or CLKB
- Other arguments (-logically_exclusive and -physically_exclusive) have the same effect



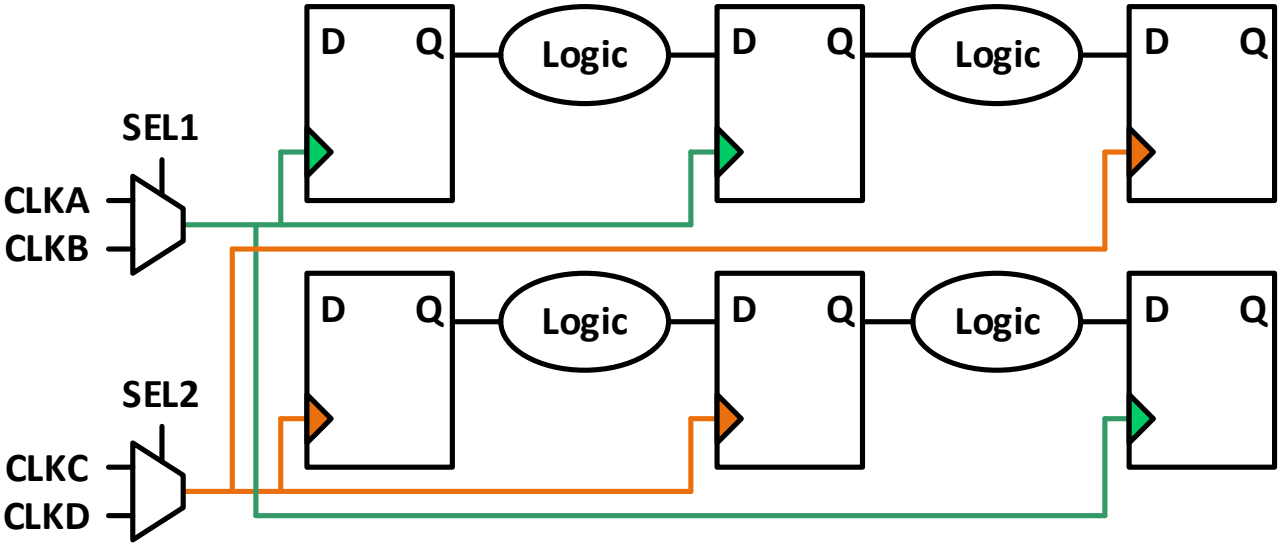
Multiple Clocks per Registers: Example 1

- DC optimization considers all of the four cases
  - CLKA - CLKA
  - CLKA - CLKB
  - CLKB - CLKA
  - CLKB - CLKB




```
create_clock -period … [get_ports CLKA]
create_clock -period … [get_ports CLKB]
set_clock_groups -logically_exclusive -group CLKA -group CLKB
```

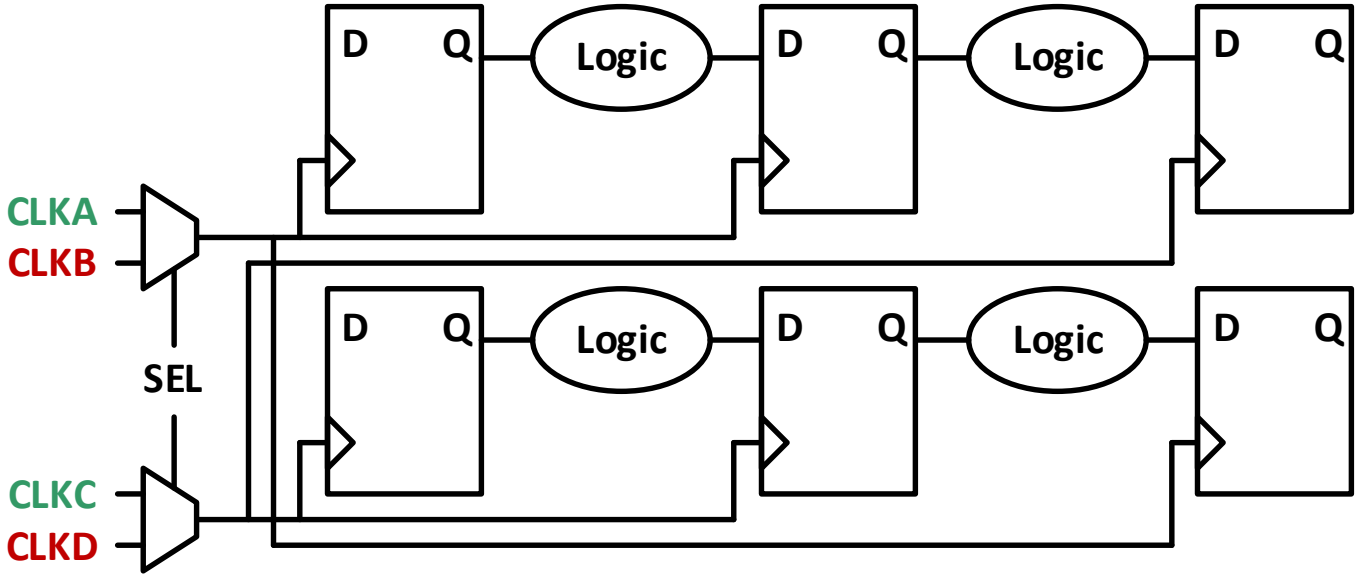
Multiple Clocks per Registers: Example 2




```
create_clock -period … [get_ports CLKA]
create_clock -period … [get_ports CLKB]
create_clock -period … [get_ports CLKC]
create_clock -period … [get_ports CLKD]
set_clock_groups -logically_exclusive -group CLKA -group CLKB
set_clock_groups -logically_exclusive -group CLKC -group CLKD
set_clock_groups -asynchronous -group {CLKA CLKB} -group {CLKC CLKD}
```



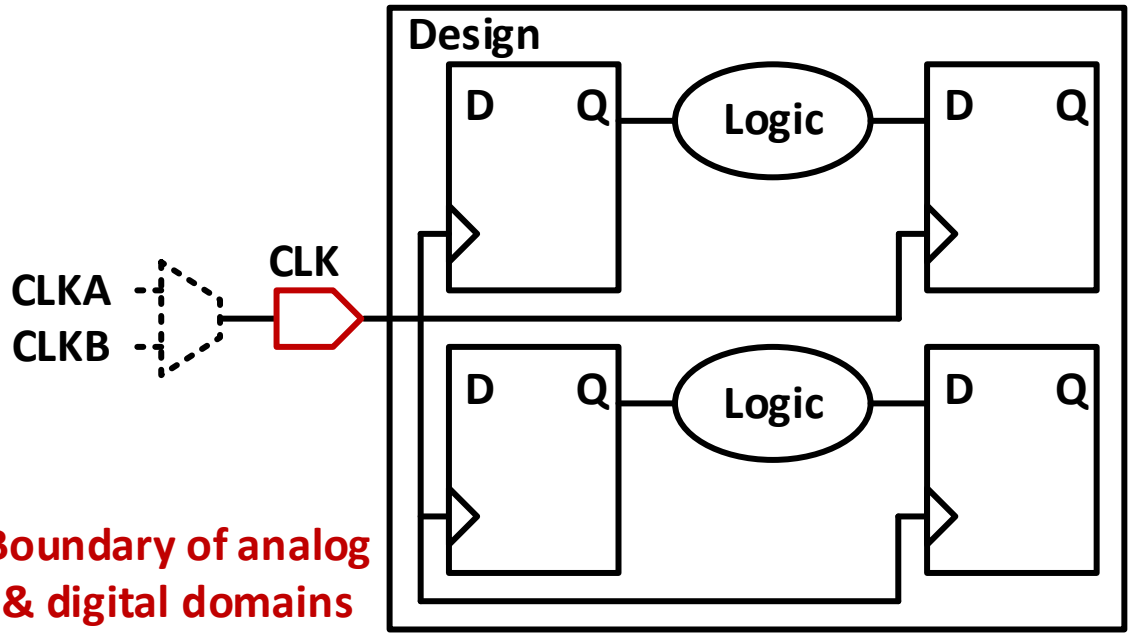
Multiple Clocks per Registers: Example 3




```
create_clock -period … [get_ports CLKA]
create_clock -period … [get_ports CLKB]
create_clock -period … [get_ports CLKC]
create_clock -period … [get_ports CLKD]
set_clock_groups -logically_exclusive -group {CLKA CLKC} -group {CLKB CLKD}
set_clock_groups -asynchronous -group CLKA -group CLKC
set_clock_groups -asynchronous -group CLKB -group CLKD
```



Multiple Clocks per Registers: Example 4




```
create_clock -period 10 -name CLKA        [get_ports CLK]
create_clock -period 20 -name CLKB -add [get_ports CLK]
set_clock_groups -physically_exclusive -group CLKA -group CLKB
```